

Estructuras de Datos y Algoritmos II

Jerónimo Delorenzi

2026

Índice

1. Modelo de costos	5
1.1. Notación asintótica	5
1.1.1. Orden de crecimiento	5
1.1.2. Convenciones matemáticas	6
1.2. Modelo de costos en algoritmos	7
1.2.1. Trabajo y profundidad	7
1.2.2. Paralelismo	7
1.2.3. Principio del Scheduler Voraz (Brent)	8
1.2.4. Pseudocódigo y costos de programas	8
1.3. Divide and Conquer	9
1.3.1. Trabajo y profundidad de algoritmos Divide and Conquer	9
1.3.2. Ejemplo, algoritmo MergeSort	10
2. Recurrencias	12
2.1. Método de sustitución	12
2.1.1. Definición	12
2.1.2. Ejemplo - Mergesort	12
2.1.3. Observación	13
2.2. Árboles de recurrencia	14
2.2.1. Definición	14
2.2.2. Ejemplo	14
2.3. Suavidad	15
2.3.1. Función eventualmente no decreciente	16
2.3.2. Función suave	16
2.3.3. Ejemplo - Mergesort	17
2.4. Teorema maestro	17
3. Estructuras Funcionales	19
3.1. Programación funcional	19

3.2.	Haskell	19
3.2.1.	Ventajas	19
3.2.2.	Compilar y ejecutar un archivo	19
3.2.3.	Comentarios y palabras reservadas	20
3.2.4.	Offside rule	20
3.2.5.	Operadores infijos	20
3.2.6.	Tipos	21
3.2.7.	Listas	21
3.2.8.	Tuplas	24
3.2.9.	Strings	25
3.2.10.	Expresiones condicionales	25
3.2.11.	Funciones	27
3.2.12.	Variables locales en funciones	29
3.2.13.	Recursión	29
3.2.14.	Funciones Lambda	30
3.2.15.	Polimorfismo paramétrico en funciones	30
3.2.16.	Polimorfismo ad-hoc de sobrecarga de funciones	30
3.2.17.	Clases de tipo	31
3.2.18.	Función <code>zip</code>	32
3.2.19.	Notación polaca	33
4.	Tipos en Haskell	35
4.1.	Sinónimos de tipos	35
4.2.	Declaraciones <code>data</code>	35
4.3.	Sintaxis para records/registros	36
4.4.	Constructores de tipo	37
4.4.1.	<code>Maybe</code>	37
4.4.2.	<code>Either</code>	38
4.5.	Tipos recursivos	38
4.6.	Expresión <code>case</code>	39
4.7.	Árboles	40
4.7.1.	Declaración	40
4.7.2.	Programando con árboles	40
4.7.3.	Árboles de Huffman	41
5.	Estructuras inmutables	43
5.1.	Estructuras de datos funcionales vs imperativas	43
5.1.1.	Inmutabilidad y Sharing	43
5.1.2.	Ejemplo: lista simplemente enlazada	43
5.1.3.	Ejemplo: sharing en función de listas	44
5.2.	Árboles binarios	44
5.2.1.	Árboles binarios de búsqueda (BST)	45
5.2.2.	Red-Black Tree	47
5.2.3.	Heaps	49

6. Tipos Abstractos de Datos (TADs)	52
6.1. TADs	52
6.1.1. Ejemplo - TAD de colas	52
6.1.2. Especificaciones algebraicas de un TAD	53
6.2. Modelos en un TAD	54
6.2.1. Implementaciones y costos de un TAD	54
6.2.2. TADs en Haskell	56
6.2.3. Conclusión	57
6.2.4. TADs básicos	57
6.3. Verificación de especificaciones	59
6.3.1. Razonamiento ecuacional	59
6.3.2. Patrones disjuntos	59
6.3.3. Extensionalidad	60
6.3.4. Inducción	61
6.3.5. Ejemplos	62
7. Estructuras paralelizables	66
7.1. Implementación paralela del mergesort	66
7.1.1. Recordando mergesort	66
7.1.2. Paralelizando mergesort	67
7.1.3. Mergesort sobre árboles	68
7.1.4. Profundidad de merge en mergesort sobre árboles	69
7.1.5. Función <code>rebalance</code>	70
7.2. Funciones de alto orden en árboles	71
7.2.1. Función <code>map</code> , <code>sum</code> , <code>flatten</code> y <code>reduce</code>	71
7.2.2. Ejemplo: contar longitud de la línea con más palabras en un texto	72
7.2.3. <code>mapreduce</code>	72
8. Secuencias	74
8.1. Definición	74
8.2. Operaciones en secuencias	74
8.3. Vista de secuencias como árbol	75
8.4. La operación <code>foldr</code>	75
8.5. La operación <code>reduce</code>	76
8.6. Orden de reducción de <code>reduce</code>	77
8.7. Divide and Conquer con <code>reduce</code>	78
8.8. MCSS usando <code>reduce</code>	78
8.9. La operación <code>scan</code>	79
8.9.1. MCSS usando <code>scan</code>	80
8.10. Arreglos persistentes	80
8.10.1. Costos de operaciones en arreglos persistentes	81
8.10.2. Especificación de costo de secuencias basadas en arreglos persistentes	82

8.10.3. Análisis de costos de reduce en secuencias con arreglos persistentes	82
8.10.4. Implementación de scan	84
8.11. Más funciones de alto orden para secuencias	86
8.11.1. La operación collect	86
8.11.2. Implementación de collect	87
8.11.3. operación MAP/COLLECT/REDUCE : map-reduce de Google	87

1. Modelo de costos

1.1. Notación asintótica

Cuando analizamos algoritmos para instancias grandes de su entrada, de manera que sólo el orden de crecimiento sea relevante, decimos que realizamos un **análisis asintótico** de su eficiencia.

1.1.1. Orden de crecimiento

El **orden de crecimiento** de una función indica con que velocidad aumenta el tiempo de ejecución cuando la entrada aumenta.

Para determinar el orden exacto, definimos lo siguiente.

La **notación** O da una cota superior para el crecimiento de f .

Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}$. Decimos que f tiene orden de crecimiento $O(g)$ y lo notamos $f \in O(g)$, si existen constantes $c \in \mathbb{R}^+, n_0 \in \mathbb{N}$ tales que

$$\forall n \geq n_0, \quad 0 \leq f(n) \leq c \cdot g(n)$$

La **notación** Ω da una cota inferior para el crecimiento de f .

Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}$. Decimos que f tiene orden de crecimiento $\Omega(g)$ y lo notamos como $f \in \Omega(g)$, si existen constantes $c \in \mathbb{R}^+, n_0 \in \mathbb{N}$ tales que:

$$\forall n \geq 0, \quad 0 \leq c \cdot g(n) \leq f(n)$$

La **notación** Θ da una cota ajustada para el crecimiento de f .

Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}$. Decimos que una función tiene orden de crecimiento $\Theta(g)$ y lo notamos como $f \in \Theta(g)$, si existen constantes $c \in \mathbb{R}^+, n_0 \in \mathbb{N}$ si:

$$f \in O(g) \text{ y } g \in \Omega(f)$$

Observaciones.

- Tanto la notación O , Ω , Θ describen un conjunto de funciones de la forma:

$$O(g(n)) = \{f : \mathbb{N} \rightarrow \mathbb{R} \mid \exists n_0 \in \mathbb{N} \exists c \in \mathbb{R}^+ : \forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n)\}$$

- $O(1) \subset O(\lg(n)) \subset O(n) \subset O(n \lg(n)) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$

- Podemos caracterizar la notación Θ .

Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}$. Decimos que $f \in \Theta(g)$ si y solo si existen constantes $c_1, c_2 \in \mathbb{R}^+, n_0 \in \mathbb{N}$ tales que:

$$\forall n \geq n_0, \quad 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

1.1.2. Convenciones matemáticas

Función asintóticamente no negativa.

Una función $f : \mathbb{N} \rightarrow \mathbb{R}$ se dice **asintóticamente no negativa** si existe $n_0 \in \mathbb{N}$ tal que

$$f(n) \geq 0 \quad \forall n \geq n_0$$

Funciones piso y techo.

Definimos el **piso de x** . Sea $x \in \mathbb{R}$, $\lfloor x \rfloor = \max\{n \mid n \leq x, n \in \mathbb{Z}\}$.

Definimos el **techo de x** . Sea $x \in \mathbb{R}$, $\lceil x \rceil = \min\{n \mid n \geq x, n \in \mathbb{Z}\}$.

Para cualquier $x \in \mathbb{R}$ se cumple $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$.

Funciones logaritmo.

Utilizamos la siguiente notación para logaritmos

$$\lg(n) = \log_2(n), \quad \ln(n) = \log_e(n)$$

Donde para $a, b, x, y \in \mathbb{R}^+$

- $\log_a(xy) = \log_a(x) + \log_a(y)$
- $\log_a(x^n) = n \log_a(x)$
- $\log_a(x) = \frac{\log_b(x)}{\log_b(a)}$
- $x^{\log_a(y)} = y^{\log_a(x)}$
- $\log_a\left(\frac{1}{x}\right) = -\log_a(x)$
- $\log_a(x) = \frac{1}{\log_x(a)}$

Funciones exponenciales

Sea $a \in \mathbb{R}^+$, entonces se cumplen las siguientes propiedades

- $a^{-1} = \frac{1}{a}$
- $(a^m)^n = a^{m \cdot n}$
- $a^m \cdot a^n = a^{m+n}$

Series

- $\sum_{k=0}^n a + bk = (n+1)\left(a + \frac{1}{2}bn\right)$
- $\sum_{k=0}^n ax^k = \frac{a-ax^{n+1}}{1-x}$, para $x \neq 1$.

- $\sum_{k=1}^n k = \frac{n(n+1)}{2}.$
- $\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$

1.2. Modelo de costos en algoritmos

Un **modelo de costos** formaliza los costos de un programa o un algoritmo para una entrada determinada. Definimos dos costos en particular.

1.2.1. Trabajo y profundidad

El **trabajo** (W) representa el costo secuencial de un programa, es decir, el costo de ejecutarlo con un solo procesador. Puede pensarse como el cómputo total que hace el programa.

La **profundidad** (S) representa el costo paralelo de un programa, esto es, el costo de si se ejecuta con infinitos procesadores. Si bien esto es imposible, basta con que haya disponibilidad de procesadores siempre que se necesite. Puede pensarse como la cantidad de cómputo que se hará en una rama de ejecución del programa.

Ejemplo.

Si sumamos n números, la idea más directa y sencilla es sumar los primeros $n-1$ números recursivamente y luego sumar el último. Esto nos daría $W(n) \in O(n)$ y $S(n) \in O(n)$, ya que en ningún momento podemos dividir el cómputo en varios procesadores. Esto se puede mejorar si decidimos dividir la lista de números a la mitad, realizar cada suma en paralelo y luego sumar ambos valores. Esto resultaría $W(n) \in O(n)$ y $S(n) \in O(\lg(n))$.

1.2.2. Paralelismo

El **paralelismo** es una medida de cuantos procesadores se pueden usar de forma eficiente. Su formula está dada por

$$P = \frac{W}{S}$$

Siempre se priorizará los algoritmos que mayor paralelismo brinden. Sin embargo, el paralelismo, al ser un cociente, podría aumentar incrementando el valor de W , el costo secuencial. Esto no es lo que buscamos, ya que el criterio para elegir entre varias implementaciones será escoger dentro de los que menor costo secuencial tienen, el de mayor paralelismo.

1.2.3. Principio del Scheduler Voraz (Brent)

El **scheduler** es un modulo del kernel del sistema operativo encargado de asignar a cada tarea pendiente un procesador para que este se encargue de llevarla a cabo.

Un scheduler es **voraz** si cuando hay un procesador libre y una tarea pendiente, el scheduler inmediatamente le asigna dicha tarea al procesador.

El **Principio del Scheduler Voraz** nos dice que en una máquina con scheduler voraz, el tiempo T de ejecución de un programa con trabajo W , profundidad S y p procesadores reales se cumple que:

$$T < \frac{W}{p} + S$$

Esto es considerado una buena cota para el tiempo, y además se puede reformular la desigualdad utilizando el paralelismo P :

$$T < \frac{W}{p} + S = \frac{W}{p} + \frac{W}{P} = \frac{W}{p} \left(1 + \frac{p}{P}\right)$$

Si $p \ll P$, entonces $\frac{p}{P}$ tiende a 0 y luego obtenemos una cota óptima para el tiempo. Estas cotas valen bajo supuestos de un scheduler voraz y de baja latencia de comunicación entre procesadores o en red.

1.2.4. Pseudocódigo y costos de programas

Se usa un modelo de costos basado en lenguajes para expresar los costos de un programa. En este caso se utilizara el análisis asintótico, ya que solo interesa cotas para los costos de los programas.

Como queremos que el modelo se abstraiga del hardware y el lenguaje sobre los cuales funcionen los programas, definimos el siguiente **pseudocódigo**:

- Constantes: 0, 1, 2, ..., True, False, [,]
- Operadores: +, -, *, /, <, >, &&, ||, if_then_else, ▷
- Pares ordinarios y paralelos: (3, 4), (3 + 4 || 5 + 6)
- Expresiones let: let x = 3+4 in x + x.
- Secuencias: [x * 2 | x ← xs]

Trabajo (W) del lenguaje pseudocódigo:

- $W(c) = 1$.
- $W(op\ e) = 1 + W(e)$.

- $W(e_1, e_2) = 1 + W(e_1) + W(e_2)$
- $W(e_1 || e_2) = 1 + W(e_1) + W(e_2)$.
- $W(\text{let } x = e_1 \text{ in } e_2) = 1 + W(e_1) + W(e_2(x \rightarrow Eval(e_1)))$,
($x \rightarrow Eval(e_1)$) representa reemplazar todas las ocurrencias de x en e_2 por la evaluación de la expresión e_1 .
- $W([f(x) | x \leftarrow xs]) = 1 + \sum_{x \in xs} W(f(x))$

Profundidad (S) del lenguaje pseudocódigo.

- $S(e) = 1$.
- $S(op\ e) = 1 + S(e)$.
- $S(e_1, e_2) = 1 + S(e_1) + S(e_2)$.
- $S(e_1 || e_2) = 1 + \max(S(e_1), S(e_2))$
- $S(\text{let } x = e_1 \text{ in } e_2) = 1 + S(e_1) + S(e_2(x \rightarrow Eval(e_1)))$
- $S([f(x) | x \leftarrow xs]) = 1 + \max_{x \in xs} S(f(x))$

1.3. Divide and Conquer

Divide and Conquer es una estrategia de resolución de problemas muy útil cuando se tienen problemas cuya resolución se puede plantear en términos de una solución del mismo problema pero de tamaño más chico. Es más fácil de plantear mediante recursión, y se compone de dos partes:

- **Caso base:** el tamaño del problema es chico, y se puede dar una solución particular y específica para esta instancia del problema.
- **Caso recursivo:** se divide el problema en subproblemas recursivamente, y por último se combinan todas las soluciones en una solución al problema general.

1.3.1. Trabajo y profundidad de algoritmos Divide and Conquer

Bajo la estructura de los algoritmos de Divide and Conquer, podemos plantear el trabajo y profundidad para un problema de tamaño n :

- $W(n) = W_{\text{dividir}}(n) + \sum_{i=1}^k W(n_i) + W_{\text{combinar}}(n)$.
- $S(n) = S_{\text{dividir}}(n) + \max_{i=1} S(n_i) + S_{\text{combinar}}(n)$.

1.3.2. Ejemplo, algoritmo MergeSort

El algoritmo **MergeSort** de ordenación de listas es un ejemplo de aplicación de la estrategia Divide and Conquer:

1. Divide la lista en dos sublistas.
2. Ordena las sublistas recursivamente.
3. Junta los resultados: lista completamente ordenada.

El pseudocódigo resulta:

```
msort : [Int] → [Int]
msort [] = []
msort [x] = [x]
msort xs = let
    (ls, rs) = split xs
    (ls', rs') = (msort ls || msort rs)
in
    merge(ls', rs')

split : [Int] → ([Int], [Int])
split [] = []
split [x] = ([x], [])
split(x:y:zs) = let
    (xs, ys) = split zs
in
    (x:xs, y:ys)

merge : ([Int], [Int]) → [Int]
merge([], ys) = ys
merge(xs, []) = xs
merge(x:xs, y:ys) = if x <= y
                    then x:merge(xs, y:ys)
                    else y:merge(x:xs, ys)
```

De lo anterior, podemos plantear las recurrencias de trabajo, donde n es la longitud de la lista:

- $W_{\text{msort}}(0) = c_0$.
- $W_{\text{msort}}(1) = c_1$.

- $W_{\text{msort}}(n) = W_{\text{split}}(n) + 2W_{\text{msort}}(\frac{n}{2}) + W_{\text{merge}}(n) + c_2.$
- $W_{\text{split}}(0) = c_3.$
- $W_{\text{split}}(1) = c_4.$
- $W_{\text{split}}(n) = W_{\text{split}}(n-2) + c_5.$
- $W_{\text{merge}}(0) = c_6.$
- $W_{\text{merge}}(n) = W_{\text{merge}}(n-1) + c_7.$

Por ahora, asumamos que $W_{\text{split}} \in O(n)$ y $W_{\text{merge}} \in O(n)$. Luego, juntando estos dos trabajos y resumiendo con el valor n que multiplica a la constante, resulta que para la función **msort**:

$$W_{\text{msort}}(n) = 2W_{\text{msort}}\left(\frac{n}{2}\right) + c_3n \in O(n \lg(n))$$

De forma similar, podemos plantear las recurrencias de profundidad:

- $S_{\text{msort}}(0) = k_0.$
- $S_{\text{msort}}(1) = k_1.$
- $S_{\text{msort}}(n) = S_{\text{split}}(n) + S_{\text{msort}}(\frac{n}{2}) + S_{\text{merge}}(n) + k_2.$
- $S_{\text{split}}(0) = k_3.$
- $S_{\text{split}}(1) = k_4.$
- $S_{\text{split}}(n) = S_{\text{split}}(n-2) + k_5.$
- $S_{\text{merge}}(0) = k_6.$
- $S_{\text{merge}}(n) = S_{\text{merge}}(n-1) + k_7.$

Por ahora, asumamos que $S_{\text{split}} \in O(n)$ y $S_{\text{merge}} \in O(n)$. Luego, juntando estas dos profundidades y resumiendo con el valor n que multiplica a la constante, resulta que para la función **msort**:

$$S_{\text{msort}}(n) = S_{\text{msort}}\left(\frac{n}{2}\right) + k_3n \in O(n)$$

2. Recurrencias

Una **recurrencia** es una función definida en términos de sí misma, es decir, se define recursivamente.

Al calcular el trabajo (W) y profundidad (S) con funciones recursivas siempre surgen recurrencias, por lo que la idea sería poder obtener una ley exacta para una recurrencia, pero como no siempre es sencillo encontrar una expresión cerrada para la recurrencia, buscamos dar cotas para esta misma. Estas cotas pueden ser obtenidas por diferentes métodos.

2.1. Método de sustitución

2.1.1. Definición

El **método de sustitución** sirve para probar que una recurrencia cumple una cota. Esta cota es obtenida 'adivinándola'. Luego, mediante la inducción matemática debemos probar que es válida para los naturales.

Podemos decir que la sustitución consiste en los siguientes pasos:

1. Adivinar la forma de la solución.
2. Probar que la forma es correcta utilizando inducción matemática.

2.1.2. Ejemplo - Mergesort

La recurrencia del trabajo del mergesort consistía en

- $W(0) = c_0$
- $W(1) = c_1$
- $W(n) = 2W(\lfloor \frac{n}{2} \rfloor) + c_2n$

Sospechamos que la cota para mergesort es $n \lg(n)$. Vamos a probar entonces por inducción que $W(n) \in O(n \lg(n)) \forall n \in \mathbb{N}$.

- Caso inductivo. Supongamos que existe $c \in \mathbb{R}^+$ tales que $0 \leq W(k) \leq ck \lg(k)$ para $k < n$ (HI).

$$\begin{aligned} W(n) &= 2W\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c_2n \stackrel{HI}{\leq} 2\left(c \left\lfloor \frac{n}{2} \right\rfloor \lg\left(\left\lfloor \frac{n}{2} \right\rfloor\right)\right) + c_2n \\ &\leq 2c \frac{n}{2} \lg\left(\frac{n}{2}\right) + c_2n \\ &= cn(\lg(n) - \lg(2)) + c_2n \\ &= cn(\lg(n) - 1) + c_2n \end{aligned}$$

$$= cn \lg(n) - cn + c_2 n \leq cn \lg(n)$$

La última igualdad ocurre si $-cn + c_2 n \leq 0 \Rightarrow -c + c_2 \leq 0 \Rightarrow c_2 \leq c$. Luego, el caso inductivo vale siempre que $c \geq c_2$.

■ Caso base:

- Si $n = 0$, $W(0) \leq c \cdot 0 \cdot \lg(0)$. Como $\lg(0)$ no está definido, no vale este caso base.
- Si $n = 1$, $W(1) \leq c \cdot \lg(1)$. Como $\lg(1) = 0$, no existe constante c tal que valga la desigualdad, por lo que este caso base no vale.
- No valen los casos bases para 0 y 1, pero la notación bigO solo nos pide que la desigualdad comience a valer a partir de un n_0 . Probemos a partir de 2 y 3, usamos los casos bases porque el piso $\lfloor \frac{n}{2} \rfloor$ recae en 0 o 1, pero como no valen, recaerán en 2 y 3.
- Si $n = 2$, $W(2) \leq c \cdot 2 \cdot \lg(2)$

$$W(2) = 2W\left(\left\lfloor \frac{2}{2} \right\rfloor\right) + 2c_2 = 2W(1) + 2c_2 = 2c_1 + 2c_2 = 2(c_1 + c_2)$$

Luego, debemos elegir c tal que $W(2) = 2(c_1 + c_2) \leq 2c \lg(2) = 2c$. Es decir, un c tal que $2(c_1 + c_2) \leq 2c \Rightarrow c_1 + c_2 \leq c$, para que valga el caso base $n = 2$.

- Si $n = 3$, queremos ver si $W(3) \leq c \cdot 3 \cdot \lg(3)$.

$$W(3) = 2W\left(\left\lfloor \frac{3}{2} \right\rfloor\right) + 3c_2 = 2W(1) + 3c_2 = 2c_1 + 3c_2$$

Luego, debemos elegir c tal que $W(3) = 2c_1 + 3c_2 \leq 3c \lg(3)$, es decir, un c tal que $\frac{2c_1 + 3c_2}{3 \lg(3)} \leq c$.

Tomando todos los casos bases y el inductivo, tenemos que si tomamos

$$c \geq \max\left\{c_2, c_1 + c_2, \frac{2c_1 + 3c_2}{3 \lg(3)}\right\}$$

concluimos que $\forall n \geq 2, W(n) \leq cn \lg(n)$ y por lo tanto $W(n) \in O(n \lg(n))$.

2.1.3. Observación

Como adivinanza de una solución $O(n \lg(n))$ utilizamos la función $f(n) = cn \lg(n)$ para una constante c arbitraria. Obviamente $f(n) \in O(n \lg(n))$, pero también, podemos utilizar cualquier función perteneciente a $O(n \lg(n))$ para las pruebas. Por ejemplo:

- $f(n) = cn \lg(n) + c_1 n$.
- $f(n) = cn \lg(n) - c_2$.
- $f(n) = cn \lg(n) - c_1 n + c_2$.

El sumar o restar elementos de menor orden puede ser crucial para poder terminar la prueba.

2.2. Árboles de recurrencia

2.2.1. Definición

Este método es útil para hallar estimaciones o posibles cotas a una recurrencia. Consiste en dibujar un árbol, donde el nodo raíz se tiene el costo de la recurrencia para la entrada n y hay una rama por cada llamada recursiva.

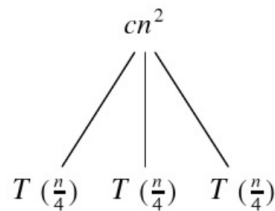
Cada rama de una llamada recursiva, a su vez, se expande en su propio árbol, y se procede así hasta llegar a un caso base, donde el árbol no se expande más. Por cada nivel del árbol se suma el total de operaciones, y estos a su vez se suman para dar el costo total de la recurrencia.

2.2.2. Ejemplo

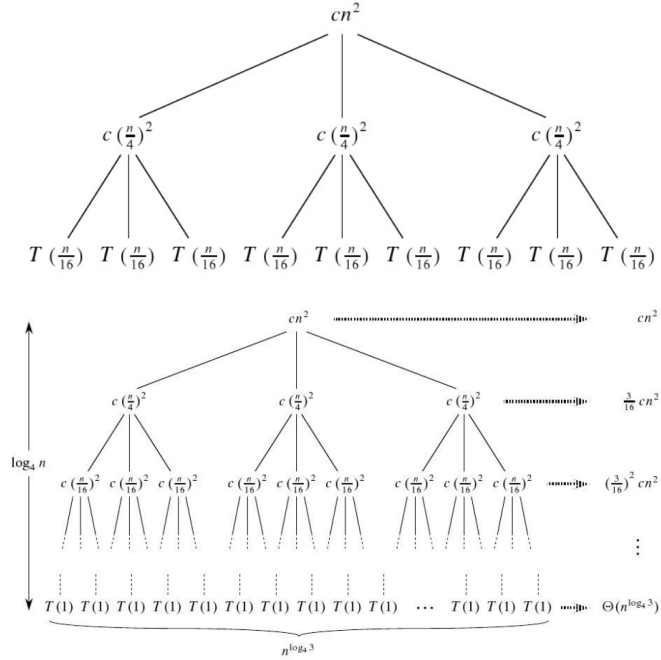
Consideremos la siguiente recurrencia:

- $T(1) = c_1$.
- $T(n) = 3T(\frac{n}{4}) + cn^2$.

Realizamos un árbol que en su raíz tiene el costo para el n inicial, con ramas indicando cada una de las llamadas recursivas. Las ramas correspondientes



a las llamadas recursivas se expanden, ahora tienen el costo correspondiente a cada llamada y generan nuevas hojas. Se sigue expandiendo el árbol hasta llegar a un caso base ($T(1)$ por ejemplo). En cada nivel se suma el total de operaciones por nivel. El costo total de la recurrencia es la suma de todos los niveles. En este ejemplo, el nivel 1 suma cn^2 a la suma total. El segundo nivel suma $\frac{3}{16}cn^2$, el tercero $(\frac{3}{16})^2cn^2$.



Luego, tenemos $\lg_4(n)$ niveles y $n^{\lg_4(3)}$ hojas. Por lo tanto, el último nivel es de orden $O(n^{\lg_4(3)})$.

Finalmente, la suma obtenida se manipula algebraicamente para llegar al resultado. Si somos prolijos, obtenemos una solución exacta, caso contrario, nos da un candidato para usar el metodo de sustitución.

$$\begin{aligned}
 T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \dots + \left(\frac{3}{16}\right)^{\lg_4(n-1)} cn^2 + kn^{\lg_4(3)} \\
 &= \sum_{i=0}^{\lg_4(n-1)} \left(\frac{3}{16}\right)^i cn^2 + kn^{\lg_4(3)} = \frac{\left(\frac{3}{16}\right)^{\lg_4(3)} - 1}{\left(\frac{3}{16} - 1\right)} cn^2 + kn^{\lg_4(3)} \in O(n^2)
 \end{aligned}$$

Luego de obtener la cota con el árbol de recurrencia se debe utilizar el método de sustitución para probarlo.

Observación.

Si en una definición de recurrencia no se menciona el caso base, suponer que es una constante, por ejemplo, $T(1) = c_1$.

2.3. Suavidad

En el planteo de mergesort se omitieron los pisos ($\lfloor \cdot \rfloor$) y techos ($\lceil \cdot \rceil$), siendo que ambos corresponden ya que n podría ser impar, quedando una mitad de lista de longitud $\lfloor \frac{n}{2} \rfloor$ y otra de longitud $\lceil \frac{n}{2} \rceil$

Entonces para omitir los pisos y los techos debe ocurrir que $n = b^k$ (n potencia de b), así entonces, $\frac{n}{b} = \lfloor \frac{n}{b} \rfloor = \lceil \frac{n}{b} \rceil$. No alcanza con que n sea múltiplo de b , debe ser potencia.

2.3.1. Función eventualmente no decreciente

Una función $f : \mathbb{N} \rightarrow \mathbb{R}^+$ es **eventualmente no decreciente** si

$$\exists N \in \mathbb{N} : f(n) \leq f(n+1) \quad \forall n \geq N$$

Es decir, una función que a partir de cierto N no decrece, se mantiene constante o crece.

2.3.2. Función suave

Una función $f : \mathbb{N} \rightarrow \mathbb{R}^+$ es **b -suave** si

- f es eventualmente no decreciente.
- $f(bn) \in O(f(n))$.

Es decir, el crecimiento de f está acotado y por lo tanto no crece exponencialmente rápido.

Si f es b -suave para todo $b \in \mathbb{N}$ entonces f es una función **suave**.

Ejemplo.

Las funciones $n^2, n^r, n \lg(n)$ son funciones suaves. Por otro lado, las funciones $n^{\lg(n)}, 2^n, n!$ no lo son.

Regla de suavidad-Si f es una función suave y g eventualmente no decreciente, entonces para todo $b \geq 2$

$$g(b^k) \in \Theta(f(b^k)) \Rightarrow g(n) \in \Theta(f(n))$$

Esta regla nos sirve para eliminar los pisos y techos en las recurrencias, pues si en vez de considerar n cualquiera, consideramos potencias de b , es decir $n = b^k$, entonces podemos omitir pisos y techos de nuestro análisis, ya que $\frac{n}{b} = \lfloor \frac{n}{b} \rfloor = \lceil \frac{n}{b} \rceil$.

Luego, la regla de suavidad nos dice que si para nuestro análisis consideramos exclusivamente a potencias de b , podemos omitir pisos y techos; y si el resultado al que llegamos es una función suave, entonces nuestro análisis es correcto para cualquier entrada.

2.3.3. Ejemplo - Mergesort

Podemos resolver el caso del mergesort.

$$W_{\text{msort}}(n) = \begin{cases} c_0 & \text{si } n = 0 \\ c_1 & \text{si } n = 1 \\ W_{\text{msort}}(\lfloor \frac{n}{2} \rfloor) + W_{\text{msort}}(\lceil \frac{n}{2} \rceil) + W_{\text{split}}(n) + W_{\text{merge}}(n) & \text{si } n > 1 \end{cases}$$

Probemos primero que $W_{\text{msort}}(n)$ es eventualmente no decreciente, para luego, por regla de suavidad, eliminar pisos y techos.

Lema. $W_{\text{msort}}(n)$ es eventualmente no decreciente.

Demostración.

Probemos por inducción sobre los naturales. Supongamos que vale $W_{\text{msort}}(k) \leq W_{\text{msort}}(k+1)$ con $k+1 \leq n$. Luego

$$\begin{aligned} W_{\text{msort}}(n) &= W_{\text{msort}}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + W_{\text{msort}}\left(\left\lceil \frac{n}{2} \right\rceil\right) + O(n) \\ &\stackrel{HI}{\leq} W_{\text{msort}}\left(\left\lfloor \frac{n+1}{2} \right\rfloor\right) + W_{\text{msort}}\left(\left\lceil \frac{n+1}{2} \right\rceil\right) + O(n) \\ &= W_{\text{msort}}(n+1) \end{aligned}$$

Vemos entonces que $W_{\text{msort}}(n) \leq W_{\text{msort}}(n+1)$. Como caso base podemos ver que $W_{\text{msort}}(2) \geq W_{\text{msort}}(1)$ y concluimos que $W_{\text{msort}}(n)$ es eventualmente no decreciente. ■

Siguiendo con la recurrencia de $W_{\text{msort}}(n)$, notemos que si $n = 2^k$, entonces podemos eliminar pisos y techos. Tenemos que

$$\begin{aligned} W_{\text{msort}}(n) &= W_{\text{msort}}\left(\frac{n}{2}\right) + W_{\text{msort}}\left(\frac{n}{2}\right) + W_{\text{split}}(n) + W_{\text{merge}}(n) \\ &= 2W_{\text{msort}}\left(\frac{n}{2}\right) + W_{\text{split}}(n) + W_{\text{merge}}(n) \\ &= 2W_{\text{msort}}\left(\frac{n}{2}\right) + O(n) \in O(n \lg(n)) \end{aligned}$$

Luego, como la función $f(n) = n \lg(n)$ es suave y $W_{\text{msort}}(n)$ es eventualmente no decreciente, entonces por regla de suavidad podemos concluir que $W_{\text{msort}}(n) \in O(n \lg(n))$.

2.4. Teorema maestro

Dados $a \geq 1, b > 1$ y la recurrencia

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

donde n es el tamaño del problema, a es la cantidad de subproblemas en los que se divide el problema original, $\frac{n}{b}$ es el tamaño del subproblema y $f(n)$ es el costo de realizar las llamadas recursivas. Entonces tenemos las siguientes cotas

$$T(n) \in \begin{cases} \Theta(n^{\lg_b(a)}) & \text{si } \exists \epsilon > 0 : f(n) \in O(n^{\lg_b(a)-\epsilon}) \\ \Theta(n^{\lg_b(a)} \lg(n)) & \text{si } f(n) \in \Theta(n^{\lg_b(a)}) \\ \Theta(f(n)) & \text{si } \exists \epsilon > 0 : f(n) \in \Omega(n^{\lg_b(a)+\epsilon}) \wedge \\ & \exists c < 1, N \in \mathbb{N} : af\left(\frac{n}{b}\right) \leq cf(n) \quad \forall n \geq N \end{cases}$$

Observaciones.

- El teorema maestro nos indica que término de la recurrencia tiene más peso asintótico en la ecuación. Los casos se deciden comparando $f(n)$ con $n^{\lg_b(a)}$:
 1. $n^{\lg_b(a)}$ es de mayor orden.
 2. $f(n)$ y $n^{\lg_b(a)}$ tienen el mismo orden.
 3. $f(n)$ es de mayor orden.
- No basta con que las funciones sean una más grande que la otra, sino que debe ser polinomialmente más grande.

3. Estructuras Funcionales

3.1. Programación funcional

La **programación funcional** es un estilo de programación que no usa un modelo de computación basado en máquinas sino en un lenguaje simple y elegante (λ -calculo).

3.2. Haskell

Usamos **Haskell** como lenguaje de programación funcional.

3.2.1. Ventajas

- Programas concisos.
- Sistemas de tipos poderoso.
- Funciones recursivas.
- Facilidad para probar propiedades de programas.
- Funciones de alto orden.
- Evaluación perezosa.
- Facilidad para definir DSLs.
- Efectos monádicos.

3.2.2. Compilar y ejecutar un archivo

- Para compilar y ejecutar programas en Haskell, se necesita instalar **GHC** (Glorious Haskell Compiler).
- Para escribir un código en Haskell, se usa un editor de texto.

```
-- Definicion del modulo principal.  
module Main where  
  
-- Definiciones.  
numero = 5  
lista = [1, 2, 3]
```

- Para compilar y ejecutar un archivo haskell:

```

-- Compilar el archivo con ghci.
ghci archivo.hs

-- Ejecutar el modulo.
main

-- Mostrar variables.
numero
lista

-- Salir de ghci.
:quit

```

3.2.3. Comentarios y palabras reservadas

Las palabras reservadas en Haskell son: `case`, `class`, `data`, `default`, `deriving`, `do`, `else`, `if`, `import`, `in`, `infix`, `infixl`, `infixr`, `instance`, `let`, `module`, `newtype`, `of`, `then`, `type`, `where`.

Los comentarios se escriben con un doble guion (`--`) para comentarios en línea y con `{-}` para bloques de comentarios.

```

-- Comentario en linea
{-
    Bloque de
    comentarios.
-}

```

3.2.4. Offside rule

En una serie de definiciones, cada definición debe empezar en la misma columna. Gracias a esta regla no hace falta una sintaxis explícita para agrupar definiciones.

```

a = b + c
  where
    b = 1
    c = 2
d = a + 2

```

3.2.5. Operadores infijos

Los **operadores infijos** son funciones como cualquier otra, pero que se aplican entre medio de dos argumentos. Una función se puede hacer infija con `backquotes`,

$$10 \text{ 'div' } 4 = \text{div } 10 \ 4$$

Se pueden definir operadores infijos usando algunos de los símbolos disponibles

$$a**b = (a*b)+(a+1)*(b-1)$$

La asociatividad y precedencia se indica usando `infixr` (asociatividad derecha), `infixl` (asociatividad izquierda), o `infix` (si los paréntesis deben ser obligatorios).

`infixr 6 (**)`

3.2.6. Tipos

Un tipo es un nombre para una colección de valores. Por ejemplo, `Bool` contiene los valores `True` y `False`. Entonces, escribimos `True :: Bool` y `False :: Bool`.

En general, si una expresión e tiene tipo t escribimos:

$e :: t$

En Haskell, toda expresión válida tiene un tipo. El tipo de cada expresión es calculado previo a su evaluación mediante la inferencia de tipos. Si no es posible encontrar un tipo, el compilador protestará con un error de tipo. Por ejemplo si evaluamos `(True + 4)`.

Algunos de los tipos básicos de Haskell son:

- `Bool` → booleanos.
- `Char` → caracteres.
- `Int` → enteros de precisión fija.
- `Integer` → enteros de precisión arbitraria.
- `Float` → números de punto flotante de precisión simple.

3.2.7. Listas

Una **lista** es una secuencia de valores del mismo tipo.

En Haskell, las listas se declaran utilizando corchetes `[]` para delimitar los elementos de la lista, separados por comas.

En general, `[t]` es una lista con elementos de tipo t , donde t puede ser cualquier tipo válido.

```
listaBool :: [Bool]
listaBool = [True, False, False, True]

listaPalabra :: [Char]
listaPalabra = ['h', 'o', 'l', 'a']

listaEjemplo :: [[Char]]
listaEjemplo = [['a'], ['b', 'c'], []]
```

Observaciones.

No hay restricción con respecto a la longitud de las listas.

Operaciones básicas con listas.

```
-- Concatenacion, utilizando el operador ++.
lista1 ++ lista2

-- Agregar un elemento al principio de la lista (cons), utilizando el operador (:).
1 : [2, 3, 4, 5]

-- Obtener la longitud de la lista, utilizando la funcion (length).
length [1, 2, 3, 4, 5]

-- Acceder a un elemento de la lista segun el indice. Se utiliza la funcion infija <
  lista> !! <indice>. Asi obtenemos el elemento de indice indicado.
[1, 2, 3, 4, 5] !! 2
```

Funciones de listas.

```
-- HEAD: devuelve el primer elemento de una lista.
lista = [1, 2, 3, 4, 5]
primerElemento = head lista
-- primerElemento = 1

-- TAIL: devuelve todos los elementos de una lista excepto el primero.
lista = [1, 2, 3, 4, 5]
restoLista = tail lista
-- restoLista = [2, 3, 4, 5]

-- LAST: devuelve el ultimo elemento de una lista.
lista = [1, 2, 3, 4, 5]
ultimoElemento = last lista
-- ultimoElemento = 5

-- INIT: devuelve todos los elementos de una lista excepto el ultimo.
lista = [1, 2, 3, 4, 5]
sinUltimoElemento = init lista
-- sinUltimoElemento = [1, 2, 3, 4]

-- NULL: comprueba si una lista esta vacia.
listaVacia = []
estaVacia = null listaVacia
estaVacia = True

-- REVERSE: invierte una lista
lista = [1, 2, 3, 4, 5]
listaInvertida = reverse lista
-- listaInvertida = [5, 4, 3, 2, 1]

-- TAKE: toma los primeros n elementos de una lista.
lista = [1, 2, 3, 4, 5]
primerosTresElementos = take 3 lista
-- primerosTresElementos = [1, 2, 3]

-- DROP: elimina los primeros n elementos de una lista.
lista = [1, 2, 3, 4, 5]
sinPrimerosTresElementos = drop 3 lista
-- sinPrimerosTresElementos = [4, 5]

-- ELEM: comprueba si un elemento esta presente en una lista.
```

```

lista = [1, 2, 3, 4, 5]
estaTres = elem 3 lista
-- estaTres = True

-- FILTER: filtra los elementos de una lista segun un predicado.
lista = [1, 2, 3, 4, 5]
pares = filter even lista
-- pares = [2, 4]

-- MAP: aplica una funcion a cada elemento de una lista.
lista = [1, 2, 3, 4, 5]
cuadrados = map (\x -> x^2) lista
-- cuadrados = [1, 4, 9, 16, 25]

-- FOLDR/FOLD: realiza un plegado sobre una lista, combinando los elementos usando
-- una funcion. Toma una funcion, un valor inicial y una lista. Aplica la funcion
-- acumulativa a cada elemento de la lista, partiendo desde el lado derecho (r en
-- foldr).
lista = [1, 2, 3, 4, 5]
suma = foldr (+) 0 lista
-- suma = 15

```

Listas por comprensión.

Haskell permite construir listas de forma concisa mediante la especificación de reglas de generación y filtros, es decir, generar **listas por comprensión**. La notación para generar listas por comprensión es:

$$[x^2 \mid x \leftarrow [1 \dots 5]]$$

Esta ultima sentencia genera la lista $[1, 4, 9, 16, 25]$. Notar que la expresión $x \leftarrow [1 \dots 5]$ es un generador, ya que dice como se generan los valores de x .

```

-- Genera una lista de los cuadrados del 1 al 5.
cuadrados = [x^2 | x <- [1 .. 5]]
-- [1, 4, 9, 16, 25]

-- Una lista por compresion puede tener varios generadores, separados por una coma.
[(x,y) | x <- [1, 2, 3], y <- [4, 5]]
-- [(1,4), (1,5), (2,4), (2,5), (3,4), (3,5)]

-- Un generador puede depender de un generador anterior. A estos los llamamos
-- GENERADORES DEPENDIENTES.
-- La siguiente es la lista de todos los pares (x, y) tal que x, y estan en [1 .. 3]
-- e y >= x.
[(x,y) | x <- [1 .. 3], y <- [x .. 3]]
-- [(1,1), (1,2), (1,3), (2,2), (2,3), (3,3)]

```

Patrones de listas.

Toda lista (no vacía) se construye usando el operador $(:)$ llamado **cons**, que agrega un elemento al principio de la lista.

$$[1, 2, 3, 4] = 1 : (2 : (3 : (4 : [])))$$

Por lo tanto, se pueden definir funciones usando el patrón $(x:xs)$. Este patrón solo matchea en el caso de listas no vacías. Por lo tanto, la función **head** con una lista vacía arrojaría un error.

```

head :: [a] → a
head (x:_) = x

tail :: [a] → a
tail (_:xs) = xs

```

Guardas.

Las listas por comprensión pueden incluir condiciones lógicas llamadas **guardas** para restringir o filtrar los valores producidos por los generadores anteriores. Si la guarda se evalúa como **True**, el elemento se incluye en la lista resultante; si es **False**, se descarta.

$$[x \mid x \leftarrow [1 \dots 10], \text{even } x]$$

Esta sentencia genera la lista `[2, 4, 6, 8, 10]`. El generador produce los números del 1 al 10, pero la guarda `even x` filtra la lista conservando únicamente los números pares.

```

-- Genera una lista de numeros pares del 1 al 10.
pares = [x | x <- [1 .. 10], even x]
-- [2, 4, 6, 8, 10]

-- Las guardas pueden usar funciones definidas por nosotros.
-- Por ejemplo, encontrar los divisores (factores) de un numero n:
-- (Acepta el numero x solo si el resto de n / x es cero)
factors :: Int → [Int]
factors n = [x | x <- [1 .. n], n `mod` x == 0]
-- factors 15 devuelve [1, 3, 5, 15]

-- Combinando guardas y listas, podemos definir si un numero es primo.
-- Un numero n es primo si y solo si sus unicos divisores son 1 y n.
prime :: Int → Bool
prime n = factors n == [1, n]

-- Finalmente, usamos esa funcion como guarda para generar una lista de primos:
primes :: Int → [Int]
primes n = [x | x <- [2 .. n], prime x]
-- primes 10 devuelve [2, 3, 5, 7]

```

3.2.8. Tuplas

Una **tupla** es una secuencia finita de valores de tipos (posiblemente) distintos. A diferencia de las listas, las tuplas tienen un tamaño fijo y pueden contener elementos de tipos diferentes.

En Haskell, las tuplas se declaran utilizando paréntesis `()` para delimitar los elementos de la tupla, separados por comas.

En general, $(t_1, t_2, \dots, t_n)$ es el tipo de una n -tupla cuyas componentes i tiene el tipo t_i .

```
-- miTupla :: (Bool, Bool)
miTupla = (True, True)

-- miTupla2 :: (Bool, Char, Char)
miTupla2 = (True, 'a', 'b')

-- miTupla3 :: (Char, (Bool, Char))
miTupla3 = ('a', (True, 'c'))

-- miTupla4 :: (([Char], Char), Char)
miTupla4 = ((['a', 'b'], 'c'), 'd')
```

Acceso a elementos.

En Haskell no se puede acceder directamente a los elementos de una tupla como lo haríamos en otros lenguajes utilizando índices. En su lugar, debemos utilizar **pattern matching** o funciones predefinidas.

```
-- Definicion de nuestra tupla.
miTupla = ('a', 24, True)

-- Definicion de funciones.
primero (x, _, _) = x
segundo (_, y, _) = y
tercero (_, _, z) = z

-- Usando las funciones predefinidas, validas para tuplas con dos elementos.
miTupla2 = ('a', True)
fst miTupla2
snd miTupla2
```

3.2.9. Strings

Un **string** es una lista de caracteres.

```
"Hola" :: String
"Hola" = ['H', 'o', 'l', 'a']
```

Por lo tanto, todas las funciones sobre listas son aplicables a strings.

```
cantMinusc :: String → Int
cantMinusc xs = length[x | x ← xs, isLower x]
```

3.2.10. Expresiones condicionales

If-then-else

Las **expresiones condicionales** son una forma de control de flujo que permite ejecutar diferentes bloques de código según una condición específica. En Haskell se realizan a través de **if-then-else**.

Para que la expresión condicional tenga sentido, ambas ramas de la misma deben tener el mismo tipo. Además, siempre deben tener la rama **else** para que no haya ambigüedades en caso de anidamiento

```

esPositivo :: Int → Bool
esPositivo x = if x > 0 then True else False

abs :: Int → Int
abs n = if n >= 0 then n else -n

signum :: Int → Int
signum n = if n < 0 then -1 else
            if n == 0 then 0 else 1

```

Ecuaciones con guardas.

Una alternativa a los condicionales es el uso de **ecuaciones con guardas**. En este caso `'|'` se utiliza para definir múltiples casos y `'otherwise'` se utiliza como un patrón de 'cualquier otro caso'.

```

esPositivo :: Int → Bool
esPositivo x
  | x > 0 = True
  | otherwise = False

```

Pattern Matching

El **pattern matching** es una característica que permite escribir funciones que se comportan de manera diferente según la forma de los datos de entrada. Permite descomponer estructuras de datos y tomar decisiones basadas en la forma de esas estructuras.

El pattern matching se utiliza principalmente en la definición de funciones para manejar diferentes casos según los valores de entrada. Se puede aplicar a listas, tuplas, tipos algebraicos y otros tipos de datos.

Los patrones de coincidencia se evalúan en orden. Si el primer patrón es cierto, se accede a él y termina la función. El valor `(_)` representa cualquier argumento, se suele poner al final de los patrones para usarlo como `else`.

```

-- Funcion not booleano.
not :: Bool → Bool
not False = True
not True = False

-- Funcion que determina si un entero es 0.
esCero :: Int → Bool
esCero 0 = True
esCero _ = False

-- Funcion que devuelve la longitud de una lista de cualquier tipo.
longitud :: [a] → Int
longitud [] = 0
longitud (x:xs) = 1 + longitud xs

-- Funcion AND condicional.
(and) :: Bool → Bool → Bool
True (and) True = True
True (and) False = False
False (and) True = False
False (and) False = False

```

```
-- Funcion AND condicional resumilda
(and) :: Bool → Bool → Bool
True (and) True = True
_ (and) _ = False
```

3.2.11. Funciones

Una **función** mapea valores de un tipo en valores de otro tipo. En general, una función de tipo $t_1 \rightarrow t_2$ mapea valores de tipo t_1 en valores de tipo t_2 .

Las funciones son fundamentales en Haskell ya que pueden ser pasadas como argumentos a otras funciones, devueltas como resultados y asignadas a variables

Aplicación de funciones, asociatividad y precedencia.

- En Haskell la aplicación se denota con un espacio y asocia a la izquierda.

Matemáticas	Haskell
$f(x)$	$f\ x$
$f(x, y)$	$f\ x\ y$
$f(g(x))$	$f\ (g\ x)$
$f(x, (g(y)))$	$f\ x\ (g\ y)$
$f(x)\ g(y)$	$f\ x\ * \ g\ y$

- La aplicación tiene mayor precedencia que cualquier otro operador:

$$f\ x + y = (f\ x) + y$$

- Las funciones y sus argumentos deben empezar con minúscula, y pueden ser seguidos por cero o más letras (mayúsculas o minúsculas), dígitos, guiones bajos, y apóstrofes.

Declaración de funciones.

Las funciones se definen utilizando la sintaxis:

$$f :: a \rightarrow b$$

donde f es el nombre de la función, a es el tipo de los argumentos de entrada y b el tipo del resultado de la función.

```
-- Recibe un entero y devuelve su doble.
doble :: Int → Int
doble x = x * 2
```

```

-- Funcion que mapea valores de un tipo en otro.
not :: Bool → Bool
isDigit :: Char → Bool

-- Funciones con multiples argumentos usando tuplas y listas.
add :: (Int, Int) → Int
add (x,y) = x + y

deCeroA :: Int → [Int]
deCeroA n = [0 .. n]

```

Llamada a funciones.

Las funciones se llaman simplemente escribiendo el nombre de la función seguido de los argumentos entre espacios.

```

resultado = doble 5
-- resultado = 10

```

Currificación y aplicación parcial.

La **currificación** es una técnica en la que una función que toma múltiples argumentos se convierte en una secuencia de funciones que toman un solo argumento.

En Haskell, todas las funciones son curriadas de forma predeterminada, lo que significa que podemos aplicar parcialmente una función para obtener una nueva función que espera el resto de los argumentos.

Ejemplo.

La función suma toma dos argumentos enteros y devuelve su suma. Sin embargo, en Haskell, también podemos entender esta función como una función que toma un solo argumento y devuelve otra función que toma el segundo argumento. Entonces podemos reescribir la definición de suma:

```

suma :: Int → Int → Int
suma x y = x + y

suma :: Int → (Int → Int)
suma x = λ y → x + y

```

Ahora, podemos aplicar parcialmente la función suma y definir funciones como la siguiente:

```

sumaDos :: Int → Int
sumaDos = suma 2

resultado = sumaDos 3
-- resultado = 5

```

3.2.12. Variables locales en funciones

- Clausula **where**.

La clausula **where** se utiliza para definir nombres locales dentro de una función. Estos nombres locales son visibles únicamente dentro de la función en la que se definen.

Es útil para evitar la repetición de cálculos y para mejorar la legibilidad del código al encapsular detalles de implementación dentro de la función donde son relevantes.

```
areaCirculo :: Double → Double
areaCirculo radio = pi * radioCuadrado
  where
    radioCuadrado = radio * radio
```

- Clausula **let/in**.

Las palabras **let** e **in** también se utilizan para definir nombres locales dentro de una expresión.

let se utiliza para introducir definiciones locales. Se pueden definir uno o más nombres locales, cada uno seguido por una expresión que le asigna un valor.

in se utiliza para separar las definiciones locales de la expresión principal. Indica dónde comienza la expresión principal que utiliza los nombres locales previamente definidos.

```
areaCirculo :: Double → Double
areaCirculo radio =
  let radioCuadrado = radio * radio
  in pi * radioCuadrado
```

3.2.13. Recursión

La **recursión** o **función recursiva** es una función que en su cuerpo/definición se llama a sí misma.

Este tipo de funciones tienen una entrada de cierto tamaño y en su llamado recursivo, se llaman a sí mismas con un tamaño menor de entrada. A su vez, cuentan con un caso base que hará que la función finalice.

Esta técnica se utiliza Haskell para reemplazar a los bucles **for**. Haskell admite la recursión tanto en funciones como en estructuras de datos.

```
-- Calcular factorial de un numero
factorial :: Int → Int
factorial 0 = 1
factorial n = n * factorial(n-1)
```

3.2.14. Funciones Lambda

Se pueden construir funciones sin darles nombres usando **expresiones lambda**. La sintaxis de una función lambda es la siguiente:

$$\lambda x \rightarrow x + x$$

En Haskell escribimos el símbolo λ como una barra (`\`), x es el argumento de la función y la expresión luego de la flecha es el cuerpo de la función.

```
-- Funcion que suma dos elementos.
suma :: Int → Int → Int
suma = λ x y → x + y

-- Multiplicacion de 3 numeros.
λ x y z → x * y * z

-- Evitamos darle nombre a una funcion que se usa una vez.
impares n = map f [0 .. n-1]
  where f x = x * 2 + 1

odds n = map (λ x → x * 2 + 1) [0 .. n-1]
```

3.2.15. Polimorfismo paramétrico en funciones

El **polimorfismo** se refiere a la capacidad de una función de trabajar con múltiples tipos de datos de manera genérica.

Una función es **polimórfica** si su tipo contiene variables de tipo. Es decir, es una función que puede trabajar con múltiples tipos de datos. Puede tomar argumentos de diferentes tipos y realizar las mismas operaciones independientemente del tipo de datos que reciba.

```
-- Para cualquier lista de cualquier tipo a, la funcion length que calcula la
  cantidad de elementos de una lista es la misma.
length :: [a] → Int

-- Funcion polimorfica que intercambia los elementos de una tupla.
intercambiar :: (a,b) → (b,a)
intercambiar (x,y) = (y,x)

intercambiar (1, "Hola")
-- ("Hola", 1)
intercambiar ("Mundo", True)
-- (True, "Mundo")
```

3.2.16. Polimorfismo ad-hoc de sobrecarga de funciones

Este tipo de polimorfismo es una característica que permite definir múltiples versiones de una función u operador con el mismo nombre, pero de distintos tipos de argumentos.

Por ejemplo, la función suma (+) permite sumar `Ints`, `Floats` y otros tipos numéricos. Si vemos el tipo, tenemos que

$$(+) :: \text{Num } a \Rightarrow a \rightarrow a \rightarrow a$$

Es decir, la función suma está definida para cualquier tipo que sea una instancia de la clase `Num`. A diferencia del polimorfismo paramétrico, hay una definición distinta de la función (+) para cada instancia.

En Haskell, los números y operaciones aritméticas están sobrecargadas. Por ejemplo, ¿cuál es el tipo de `3 + 2`? Podría tomar dos `Ints`, dos `Floats`, o generalizando, dos `Num`.

3.2.17. Clases de tipo

Las **clases de tipo** son un tipo de mecanismo que permite definir interfaces de tipos y proporcionar implementaciones para esas interfaces. Se utilizan para agregar restricciones sobre los tipos de datos que pueden ser utilizados con ciertas funciones u operadores.

Una clase de tipo define un conjunto de funciones o métodos que deben ser implementados para que un tipo de datos sea considerado miembro de esa clase. Esas funciones se conocen como 'métodos de clase/tipo'.

Cuando se declara una instancia de una clase de tipo para un tipo de datos específico, se proporcionan implementaciones concretas para los métodos de clase definidos en la clase de tipo.

Algunas clases de tipo.

- **Clase EQ:** define una interfaz para la igualdad entre valores. A esta clase van a pertenecer los tipos de datos que pueden ser comparables (las funciones no pueden ser comparables entonces no pertenecen).

Para que un tipo de datos sea miembro de la clase `eq`, debe proporcionar una implementación de la función `'=='` (igualdad) y `'/'='` (desigualdad).

```
class Eq a where
    (==) :: a -> a -> Bool
    (/=) :: a -> a -> Bool
```

- **Clase ORD:** son los tipos que además de ser instancias de `EQ` poseen un orden total.

```
class Ord a where
    (<), (<=), (>), (>=) :: a -> a -> Bool
    min, max :: a -> a -> a
```

- **Clase SHOW:** son los tipos de datos cuyos valores pueden ser convertidos en una cadena de caracteres.

```
class Show a where
  show :: a → a → String
```

- **Clase READ:** clase dual. Son los tipos de datos que se pueden obtener de una cadena de caracteres.

```
class Read a where
  read :: a → String → a
```

- **Clase NUM:** son los tipos de datos numéricos. Sus instancias deben implementar las funciones.

```
class Num a where
  (+), (-), (*) :: a → a → a
  negate, abs, signum :: a → a
```

- **Clase INTEGRAL:** son los tipos de datos que son Num y además implementan

```
class Integral a where
  div, mod :: a → a → a
```

- **Clase FRACTIONAL:** son los tipos de datos que son Num y además implementan

```
class Fractional a where
  (/) :: a → a → a
  recip :: a → a
```

3.2.18. Función zip

La función `zip` mapea dos listas a una lista con los pares de elementos correspondientes:

```
zip :: [a] → [b] → [(a,b)]
-- zip ['a','b','c'] [1,2,3,4] → [('a',1),('b',2),('c',3)]
```

Ejemplo usando función zip

```
-- Funcion que toma una lista y devuelve una lista de pares de los elementos
  -- adyacentes de la lista .
pairs :: [a] → [(a,a)]
pairs xs = zip xs (tail xs)
-- pairs [1,2,3,4] → [(1,2),(2,3),(3,4)]

-- Funcion que toma una lista y devuelve True si esta ordenada; False en caso
  -- contrario .
-- Genera una lista de pares con los elementos de la lista, y por cada par pregunta
  -- si x <= y, en tal caso, tendremos una lista de Booleanos [Bool] a la cual se le
  -- aplica un and generalizado.
sorted :: Ord a ⇒ [a] → Bool
sorted xs = and [x <= y | (x,y) ← pairs xs]
```

```

-- sorted [5 ,1 ,2 ,3] → False

-- Funcion que toma dos enteros y una lista , y devuelve los elementos de la lista
  desde el indice del primer argumento al otro argumento .
-- Genera una lista de pares entre la lista argumento y los naturales del 0 a n, que
  simulan ser indices , para compararlos con i.
rangeof :: Int → Int → [a] → [a]
rangeof low hi xs = [x | (x,i) ← zip xs [0..]]
-- rangeof 2 4 [1 ,2 ,3 ,4 ,5 ,6 ,7 ,8] → [3 ,4 ,5]

```

3.2.19. Notación polaca

La **notación polaca** es una forma de escribir operaciones aritméticas donde los operadores se escriben de forma prefija.

Fue inventada por Jan Lukasiewicz y tiene como ventaja que no se requiere el uso de paréntesis para definir la precedencia.

Por ejemplo, la expresión $(6 + 5) \cdot (4 - 2)$ se escribe como $+ 6 5 - 4 2$.

Para evaluar una cadena en NP en Haskell, necesitamos procesarla recursivamente. Al leer un operador, debemos evaluar dos expresiones enteras que le siguen. Por esto, utilizamos una función auxiliar que, además de devolver el valor numérico calculado, devuelve el resto de la cadena sin procesar.

```

-- Funcion principal. Llama a la funcion auxiliar para verificar que no sobre '
  basura' en el string.
np :: String → Integer
np xs = let (r, rest) = npAux xs
        in if null rest
            then r
            else error ("Error: Basura al final: " ++ rest)

-- Funcion auxiliares para determinar operaciones.
isOperator :: Char → Bool
isOperator '+' = True
isOperator '-' = True
isOperator '*' = True
isOperator '/' = True
isOperator _ = False

operator :: Char → Integer → Integer → Integer
operator '+' = (+)
operator '-' = (-)
operator '*' = (*)
operator '/' = div
operator _ = error "not an operator"

-- Funcion auxiliar npAux. Calcula el valor de un prefijo y devuelve el resto del
  string. (≠')
npAux :: String → (Integer, String)
npAux [] = error "Error: esperando mas caracteres"
-- (0, "")
npAux (x:xs) | isOperator x = let
                                (n, xs~) = npAux xs
                                (m, ys) = npAux xs~
                                in (operator x n m, ys)

```

```

| isDigit x = let
                (ns, xs^) = span isDigit(x:xs)
                in (read ns, xs^)
| isSpace x = npAux xs
| otherwise = error ("Error: encuentre un " ++ [x])

-- Ejemplo de uso:
-- np "* + 2 * 3 5 4" devuelve 68

```

4. Tipos en Haskell

Haskell tiene un **sistemas de tipo expresivo**, permitiendo definir tipos de datos de manera flexible y expresiva. Es de **tipado estático**, todos los valores y expresiones tienen un tipo conocido en tiempo de compilación. Esto significa que el tipo de cada expresión se determina antes de que se ejecute el programa, lo que ayuda a detectar errores de tipo en etapas tempranas. También tiene **inferencia de tipos**, el compilador puede inferir automáticamente muchos tipos basados en el contexto del programa. Sin embargo, dar el tipo de las funciones es ventajoso.

4.1. Sinónimos de tipos

En Haskell se puede definir un nuevo nombre para un tipo existente usando una declaración `type`. Los sinónimos de tipo hacen que ciertas declaraciones de tipos sean más fáciles de leer.

La declaración sigue de la siguiente sintaxis

```
type String = [Char]
```

Ejemplo.

```
type Pos = (Int, Int)

origen :: Pos
origen = (0,0)

izq :: Pos → Pos
izq (x,y) = (x-1, y)
```

Además, los sinónimos pueden tener parámetros, pueden anidarse, pero no pueden ser recursivos.

```
-- Con parámetros.
type Par a = (a,a)
copiar :: a → Par a
copiar x = (x,x)

-- Anidados.
type Punto = (Int, Int)
type Trans = Punto → Punto

-- No pueden ser recursivos.
type Tree = (Int, [Tree])
```

4.2. Declaraciones data

Los `data` declaran un nuevo tipo cuyos valores se especifican en la declaración.

Por ejemplo, en `data Bool = False | True`, `False` y `True` son los **constructores** del tipo `Bool` (comienzan con mayúscula). Dos constructores diferentes construyen diferentes valores de tipo.

Los valores de un nuevo tipo se usan igual que los predefinidos:

```
data Respuesta = Si | No | Desconocida
respuestas :: [Respuesta]
respuestas = [Si, No, Desconocida]
invertir :: Respuesta → Respuesta
invertir Si = No
invertir No = Si
invertir Desconocida = Desconocida
```

Los constructores son en realidad funciones de complejidad constante $O(1)$. Por esto, pueden tener parámetros:

```
data Shape = Circle Float | Rect Float Float

square :: Float → Shape
square n = Rect n n

area :: Shape → Float
area (Circle r) = pi * r^2
area (Rect x y) = x * y
```

4.3. Sintaxis para records/registros

Los **registros/records** son una extensión de Haskell que permite definir tipos de datos con campos etiquetados. Esto facilita la manipulación de datos estructurados al proporcionar nombres descriptivos para cada campo dentro de un registro. La sintaxis es la siguiente:

```
data NombreTipo = Constructor campo1 :: Tipo1, campo2 :: Tipo2, ...
```

En vez de tener que usar la declaración `data` de la siguiente forma:

```
-- Definición de tipo de datos para guardar datos de alumnos.
data Alumno = A String String int String deriving Show

-- Definimos un alumno.
juan = A "Juan" "Perez" 21 "jperez999@gmail.com"

-- Para acceder a los datos usamos funciones.
nombre :: Alumno → String
nombre (A n _ _ _) = n

apellido :: Alumno → String
apellido (A _ a _ _) = a

edad :: Alumno → Int
edad (A _ _ e _) = e

email :: Alumno → String
email (A _ _ _ m) = m
```

Utilizamos la sintaxis de records proveída por Haskell. De esta forma no tenemos que definir las proyecciones por separado, y no tenemos que acordarnos del orden de los campos:

```
-- Definición de recor.
data Alumno = A{nombre :: String, apellido :: String, edad :: Int, email :: String}

-- Definición sin importar el orden.
juan = {apellido = "Perez", nombre = "Juan", email = "jperez999@gmail.com", edad = 21}

-- Acceso a los campos.
nombreAlumno1 :: String
nombreAlumno1 = nombre juan
```

4.4. Constructores de tipo

4.4.1. Maybe

Es un tipo de dato algebraico en Haskell que se utiliza para representar valores que pueden estar o no presentes. Es útil para manejar casos en los que una función podría devolver un valor válido o valor 'nulo' (ausencia de valor). Se define de la siguiente manera:

$$\text{data Maybe } a = \text{Nothing} \mid \text{Just } a$$

- **Maybe a :** es un tipo de datos paramétrico que indica que puede contener un valor de tipo a , o puede estar vacío.
- **Nothing:** es un constructor de datos que representa la ausencia de un valor. No lleva ningún argumento.
- **Just a :** es un constructor de datos que representa la presencia de un valor. Lleva un argumento de tipo a , que es el valor que está presente.

Maybe es un constructor de tipos, ya que dado un tipo a , construye el tipo **Maybe a** . Por lo tanto, no tiene un tipo de datos sino que tiene kind. El kind de Maybe es:

$$\text{Maybe} :: * \rightarrow *$$

Un uso particular de **Maybe** es para señalar una condición de error, o hacer total una a una función parcial:

```
-- Haciendo total a la función head de listas.
safehead :: [a] → Maybe a
safehead [] = Nothing
safehead xs = Just (head xs)

-- ejemplo de función que recibe una clave y una lista de clave/valor y devuelve el
  valor asociado a la clave pasada como argumento si la encuentra en la lista.
lookup :: Eq c ⇒ c → [(c, val)] → Maybe val
lookup _ [] = Nothing
```

```
lookup k ((c,v):xs) | k == c = Just v
                  | otherwise = lookup k xs
```

4.4.2. Either

Describe un tipo que puede tener elementos de dos tipos. En sus elementos está claro de qué tipo es el elemento almacenado. Su definición es:

```
data Either a b = Left a | Right b
```

- **Either** *a b*: es un tipo de datos paramétrico que indica que puede contener un valor de tipo *a* o un valor de tipo *b*.
- **Left** *a*: constructor de datos que por lo general representa un resultado fallido. Lleva un argumento de tipo *a*, que generalmente se usa para almacenar información sobre el error.
- **Right** *b*: constructor de datos que representa un resultado exitoso. Lleva un argumento de tipo *b*, que es el valor exitoso que ha producido.

Como **Either** es un constructor de tipos, no tiene un tipo de datos sino un kind:

```
Either :: * -> * -> *
```

Either es útil para representar resultados que pueden ser uno de dos posibles valores diferentes. Por ejemplo, manejar casos en los que una función puede devolver un resultado exitoso o un error:

```
-- Función head de listas.
safehead :: [a] -> Either String a
safehead [] = Left "head de lista vacia!"
safehead xs = Right (head xs)

-- Ejemplo de función al dividir por cero.
divide :: Float -> Float -> Either String Float
divide _ 0 = Left "No se puede dividir por cero!"
divide x y = Right (x/y)

-- Ejemplos de elementos de tipo Either Bool Bool.
Left True
Left False
Right True
Right False
```

4.5. Tipos recursivos

Los **tipos recursivos** son una característica de Haskell que permite definir tipos de datos que se refieren a sí mismos de forma recursiva. Esto permite crear estructuras de datos complejas y recursivas, como árboles, listas y otras estructuras anidadas.

Para declarar los tipos recursivos usamos las declaraciones **data** nuevamente. Los tipos recursivos además pueden tener parámetros.

```
-- Tipo de datos para números naturales.
data Nat = Zero | Succ Nat

add :: Nat → Nat → Nat
add n Zero = n
add n (Succ m) = Succ(add n m)

mult :: Nat → Nat → Nat
mult n Zero = Zero
mult n (succ m) = add n (mult n m)

-- Listas definidas recursivamente. Isomorfo a las listas preddefinidas.
data List a = Nil | Cons a (List a)

to :: List a → [a]
to Nil = []
to (Cons x xs) = x : (to xs)

from :: [a] → List a
from [] = Nil
from (x:xs) = Cons x (from xs)
```

4.6. Expresión case

Es una expresión que se utiliza para hacer coincidencia de patrones en el valor de una expresión y ejecutar diferentes acciones basadas en el patrón que coincida. La sintaxis es:

```
case expression of
  patron1 → expression1
  patron2 → expression2
  ...
```

- expression: es la expresión cuyo valor queremos analizar.
- patron1, patron2, ...: patrones que se comapran con el valor de **expression**.
- expression1, expression2, ...: expresiones que se evalúan si el valor coincide con el patrón correspondiente.

Los patrones de los diferentes casos son intentados en orden. Se usa indentación para marcar un bloque de casos.

```
esCero :: Nat → Bool
esCero n = case n of
  Zero → True
  _ → False
```

4.7. Árboles

4.7.1. Declaración

Para representar diferentes tipos de **árboles**, podemos definir un tipo de datos recursivo utilizando la declaración `data`.

```
-- Árbol con información solamente en las hojas.
data T1 a = Tip a | Bin (T1 a) (T1 a)

-- Árbol con información solamente en los nodos.
data T2 b = Empty | Branch (T2 b) b (T2 b)

-- Árbol con información de tipo a en las hojas y de tipo b en los nodos.
data T3 a b = Leaf a | Node (T3 a b) b (T3 a b)

-- Árbol con información solamente en los nodos, pero que puede tener nodos con dos
  hijos o tres hijos.
data T4 a = E | N2 a (T4 a) (T4 a) | N3 a (T4 a) (T4 a) (T4 a)

-- Árbol con múltiples hijos. Los hijos de un nodo se expresan en forma de lista.
data T4 a = Rose a [T5 a]
```

4.7.2. Programando con árboles

Declaremos funciones que midan el tamaño y altura del árbol.

```
data T1 a = Tip a | Bin (T1 a) (T1 a)

size :: T1 a → Int
size (Tip _) = 1
size (Bin t1 t2) = size t1 + size t2

depth :: T1 a → Int
depth (Tip _) = 0
depth (Bin t1 t2) = 1 + max (depth t1) (depth t2)
```

Medir un árbol de tamaño n tiene complejidad $O(n)$. Podríamos hacerlo en tiempo constante $O(1)$ si lleváramos en la estructura un campo para la medida del árbol.

```
type Weight = Int
data T a = Tip Weight a | Bin Weight (T a) (T a)

-- Funciones que obtiene la medida es O(1).
weight :: T a → Weight
weight (Tip w _) = w
weight (Bin w _ _) = w

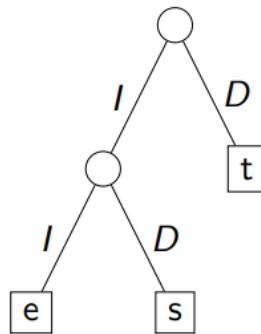
-- Queremos preservar la siguiente invariante al construir un árbol nuevo.
-- weight (Bin w t1 t2) = weight t1 + weight t2
bin :: T a → T a → T a
bin t1 t2 = let w = weight t1 + weight t2
            in Bin w t1 t2
```

4.7.3. Árboles de Huffman

Un **árbol de Huffman** es una estructura que se usa para comprimir datos. La idea es representar los caracteres de un mensaje con códigos binarios más cortos para los más frecuentes, y más largos para los menos usados. Esto permite reducir el tamaño total del mensaje.

La tabla de códigos se codifica en un árbol binario con información en las hojas. Se asignan los valores 0 o 1 (o (I) izquierda y (D) derecha) según las ramas de cada bifurcación, si vamos a la izquierda agregamos un 0, si vamos a la derecha agregamos un 1.

La secuencia `test` nos da el siguiente árbol, con los códigos correspondientes $t = 1$, $s = 01$, $e = 00$ o $t = D$, $s = ID$, $e = II$.



Representación de árboles de Huffman.

```
type Weight = Int
data T a = Tip Weight a | Bin Weight (T a) (T a)
```

Decodificación de árboles de Huffman.

Leer una secuencia de símbolos (camino en un árbol) para recuperar el mensaje original.

```
-- Representar los pasos (Izquierda o Derecha). Un camino es una lista de pasos.
data Step = I | D deriving Show
type Camino = [Step]

-- Función que sigue un camino completo hasta llegar a una hoja, y devuelve el valor
-- de esa hoja.

trace1 :: T a → Camino → a
trace1 (Tip _ x) [] = x
trace1 (Bin _ t1 t2) (I:cs) = trace1 t1 cs
trace1 (bin _ t1 t2) (D:cs) = trace1 t2 cs

-- Esta función recorre parcialmente el árbol hasta encontrar una hoja, y devuelve:
-- (el valor de esa hoja a, el resto del camino que quedo sin recorrer).
-- Es útil para decodificar una secuencia larga de bits (pasos), y queremos ir "
-- consumiendo" de a partes el camino.
```

```

trace :: T a → Camino (a, Camino)
trace (Tip _ x) resto = (x, resto)
trace (Bin _ t1 t2) (I:cs) = trace t1 cs
trace (Bin _ t1 t2) (D:cs) = trace t2 cs

-- Esta es la función principal de decodificación. Toma un árbol de Huffman y una
-- lista de pasos, y va decodificando uno a uno los símbolos hasta que se termina
-- el camino.
decodexs :: T a → Camino [a]
decodexs t ps = case trace t ps of
    (a, []) → [a]
    (a, ps^_) → a:decodexs t ps^_

```

Codificación de árboles de Huffman.

Asignarle un camino único a cada símbolo según su frecuencia en la palabra.

```

-- Codifica una lista de valores en un único camino.
codexs :: Eq a ⇒ T a → [a] → Camino
codexs t xs = concat (map (codex t) xs)

codex :: Eq t ⇒ T t → t → Camino
codex t x = head (go t x)
    where go (Tip _ y) x | x == y = [[]]
        | otherwise = []
        go (Bin _ t1 t2) x = map (I:)(go t1 x) ++ map (D:)(go t2 x)

```

5. Estructuras inmutables

5.1. Estructuras de datos funcionales vs imperativas

Muchos de los algoritmos tradicionales están pensados para **estructuras efímeras**. Este tipo de estructura soportan una sola versión de sí misma, y los cambios en ella son destructivos. Las estructuras efímeras están relacionadas a la programación secuencial e imperativa.

Por otro lado están las **estructuras persistentes** o **inmutables**. Las estructuras persistentes soportan varias versiones de si misma, es decir, la estructura no sufrirá cambios, y también son fácilmente paralelizables. El uso de este tipo de estructuras tiene un costo:

- Se deben adaptar todos los algoritmos y estructuras al modelo inmutable (en caso de ser posible).
- Hay ciertas cotas de las estructuras efímeras que no siempre se van a poder alcanzar.

5.1.1. Inmutabilidad y Sharing

En un lenguaje funcional puro, todas las estructuras son **inmutables**. Como dijimos antes, al realizar cambios, este tipo de estructura no se destruye, sino que los nodos necesarios se copian en memoria y esta copia es la que es modificada. En programación funcional, la **inmutabilidad** es una característica fundamental, donde los datos se tratan como valores constantes que no cambian con el tiempo.

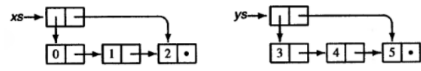
Una característica que usa Haskell cuando alguna parte de la estructura no cambia es el **sharing**. Esto permite reusar partes de una estructura en versiones modificadas y así evitar redundancia en copiado de memoria. Por esto, es necesario el manejo automático de la memoria (garbage collection) ya que es difícil determinar cuando un dato es descartado de la memoria, y también provee un nivel más de abstracción.

5.1.2. Ejemplo: lista simplemente enlazada

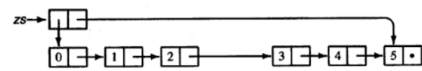
La concatenación de **listas simplemente enlazadas efímeras** es

$$zs = xs ++ ys$$

donde se destruyen las listas **xs** e **ys** pero la operación es $O(1)$.

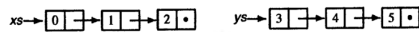


(a) Listas xs e ys

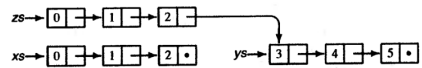


(b) Concatenación de listas efímeras

Por otro lado, la concatenación de **listas simplemente enlazadas inmutables** no destruye las listas, sino que copia xs y hace que el último nodo apunte al primero de xs . Así tendremos las tres listas xs , ys , zs , donde zs se obtuvo de copiar los nodos de xs pero comparte los nodos de ys (sharing). En este caso, la operación de copiar todos los nodos de xs en zs es del orden de $O(|xs|)$.



(a) Listas xs e ys



(b) Concatenación de listas inmutables

5.1.3. Ejemplo: sharing en función de listas

La siguiente función que modifica un solo elemento de una lista utiliza sharing, copia los nodos hasta la posición que queremos modificar, modifica el valor en el último nodo copiado, y este apunta al siguiente en la lista original.

```
update :: [a] -> Int -> a -> [a]
update [] _ _ = []
update (x:xs) 0 x^ = x^:xs
update (x:xs) i x^ = x:(update xs (i-1) x^)
```

5.2. Árboles binarios

Un **árbol binario** es un árbol en el que cada nodo tiene exactamente dos hijos. En Haskell representamos un árbol binario con la siguiente definición recursiva:

```
data Bin a = Hoja | Nodo (Bin a) a (Bin a)
```

Algunos ejemplos de funciones elementales para árboles binarios son:

```
-- Determina si un dato está en un árbol binario.
member :: Eq a => a -> Bin a -> Bool
member a Hoja = False
member a (Nodo l b r) = (a == b) || member a l || member a r

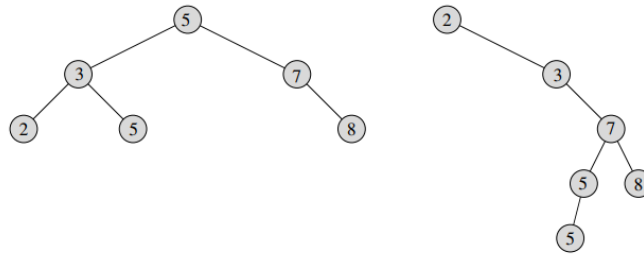
-- Recorrido inorder de un árbol binario.
inorder :: Bin a -> [a]
inorder Hoja = []
inorder (Nodo l a r) = inorder l ++ [a] ++ inorder r
```

La complejidad de `member` es $O(n)$ en el peor caso ya que recorre las n hojas.

5.2.1. Árboles binarios de búsqueda (BST)

Un **árbol binario de búsqueda** es un árbol binario t tal que

- o bien t es una hoja.
- o bien t es un Nodo l a r , y se cumple que:
 - l y r son árboles binarios de búsqueda.
 - si y es una clave en algún nodo de l entonces $y \leq a$.
 - si y es una clave en algún nodo de r entonces $a < y$.



Operaciones sobre BSTs.

```

-- Definición de BST.
data Bin a = Hoja | Nodo (Bin a) a (Bin a)

-- Función que determina si un elemento pertenece a un BST.
member :: Ord a => a -> Bin a -> Bool
member a Hoja = False
member a (Nodo l a r) | a == b = True
                      | a < b = member a l
                      | otherwise = member a r

-- Recorrido inorder de un BST.
inorder :: Bin a -> [a]
inorder Hoja = []
inorder (Nodo l a r) = inorder l ++ [a] ++ inorder r

-- Encontrar el valor mínimo de un BST.
minimum :: Bin a -> a
minimum (Nodo Hoja a _) = a
minimum (Nodo l a _) = minimum l

-- Encontrar el valor máximo de un BST.
maximum :: Bin a -> a
maximum (Nodo _ a Hoja) = a
maximum (Nodo _ a r) = maximum r

-- Chequear si un árbol es un BST (si cumple las condiciones invariantes).
checkBST :: Ord a => BST a -> Bool
checkBST Hoja = True
checkBST (Nodo Hoja a Hoja) = True
checkBST (Nodo Hoja a r) = a < minimum r && checkBST r
checkBST (Nodo l a Hoja) = a >= maximum l && checkBST l
checkBST (Nodo l a r) = a >= maximum l && a < minimum r && checkBST l && checkBST r
  
```

Teorema. Si t es un BST, entonces `member`, `minimum`, `maximum` son de Orden $O(h)$, donde h es la altura de t .

```
-- Insertar elemento en un BST (mantener invariante). Recorremos el árbol hasta
  encontrar una hoja, que transformamos en un nuevo nodo.
insert :: Ord a => a -> Bin a -> Bin a
insert a Hoja = Nodo Hoja a Hoja
insert a (Nodo l b r) | a <= b = Nodo (insert a l) b r
                      | otherwise = Nodo l b (insert a r)
```

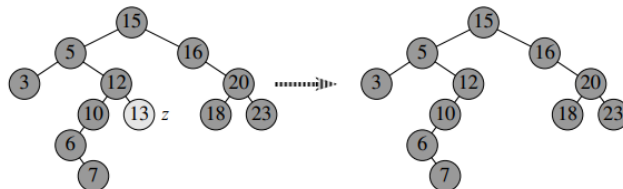
Borrado de elementos en un BST.

```
delete :: Ord a => a -> Bin a -> Bin a
delete _ Hoja = Hoja
delete a (Nodo l b r)
  | a < b = Nodo (delete a l) b r
  | a > b = Nodo l b (delete a r)
  | otherwise =
    case (l, r) of
      (Hoja, Hoja) -> Hoja
      (Hoja, _) -> r
      (_, Hoja) -> l
      (_, _) -> let y = minimum r
                  in Nodo l y (delete y r)
```

Una vez encontrado el elemento en un árbol binario de búsqueda, tendremos que hacer 3 consideraciones

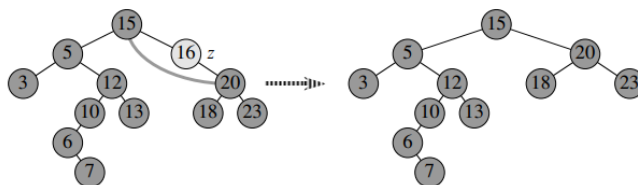
1. El nodo tiene hojas como subárboles.

```
case (l,r) of
  (Hoja, Hoja) -> Hoja
```



2. El nodo tiene un solo subárbol con datos.

```
case (l, r) of
  (Hoja, _) -> r
  (_, Hoja) -> l
```

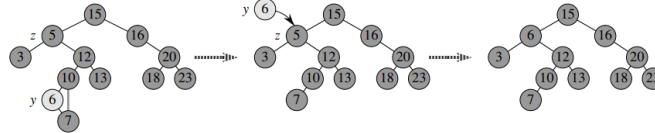


3. El nodo tiene dos subárboles con datos.

```

case (l, r) of
  (_, _) → let y = minimum r
           in Nodo l y (delete y r)

```



5.2.2. Red-Black Tree

En los árboles binarios o árboles binarios de búsqueda las operaciones de búsqueda, inserción y borrado son de orden de la altura del árbol. En el mejor de los casos son $O(\lg n)$, pero en el peor de los casos pueden ser $O(n)$, por ejemplo, al insertar datos ordenados, el árbol degenera en una lista.

La solución es mantener el **árbol balanceado**. Para eso existen diferentes estructuras como puede ser AVL. En este caso veremos la estructura **Red-Black Trees**.

Los **Red-Black Trees** son árboles binarios de búsqueda con nodos coloreados rojos o negros, que cumplen las siguientes invariantes para mantener el árbol balanceado:

1. **INV 1:** ningún nodo de color rojo tiene un hijo rojo.
2. **INV 2:** todos los caminos de la raíz a una hoja tienen la misma cantidad de hijos negros (altura negra).

Observaciones.

- En un RBT, el camino más largo es a lo sumo el doble que el camino más corto. En el peor caso el camino más corto estará formado por todos los nodos negros y el más largo intercalará entre negro y rojo. Si no se cumpliera esto, el camino más largo tendría más nodos negros o un par de nodos padre hijo ambos de color rojo.
- En un RBT la altura es $O(\lg n)$, esto quiere decir que los Red-Black Trees son árboles balanceados. Notemos que, si k es la altura negra del árbol. Entonces todos los caminos tienen k nodos negros, y en el mejor de los casos no habría nodos rojos, dando lugar a la desigualdad $n \geq 2^k - 1$. Luego $\lg(n + 1) \geq k$. Ahora, por (1), si h es la altura del árbol, $h \leq 2k$, luego $h \leq 2k \leq 2 \lg(n + 1) \Rightarrow h \in O(\lg n)$.

La definición de un RBT es la siguiente:

```
data Color = R | B
data RBT a = E | T Color (RBT a) a (RBT a)
```

Operaciones sobre RBTs.

```
-- Tipos de datos para Red-Black Trees.
data Color = R | B
data RBT a = E | T Color (RBT a) a (RBT a)

-- Función que determina si un elemento pertenece a un RBT.
member :: Ord a => a -> RBT a -> Bool
member a E = False
member a (T _ l b r) | a == b = True
                      | a < b = member a l
                      | otherwise = member a r
```

Inserción y rebalanceo de RBTs.

```
insert :: Ord a => a -> RBT a -> RBT a
insert a t = makeBlack (ins x t)
  where
    ins :: Ord a => a -> RBT a -> RBT a
    ins a E = T R E a E
    ins a (T c l b r)
      | a < b = balance c (ins a l) b r
      | a > b = balance c l b (ins a r)
      | otherwise = T c l b r

    makeBlack :: RBT a -> RBT a
    makeBlack E = E
    makeBlack (T _ l a r) = T B l a r

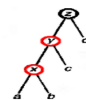
-- Rebalanceo de Red-Black Trees para arreglar posible violación del invariante 1:
-- que un nodo rojo tenga un hijo rojo, B - R - R.
balance :: Color -> RBT a -> a -> RBT a -> RBT a
balance B (T R (T R a x b) y c) z d = T R (T B a x b) y (T B c z d)
balance B (T R a x (T R b y c)) z d = T R (T B a x b) y (T B c z d)
balance B a x (T R (T R b y c) z d) = T R (T B a x b) y (T B c z d)
balance B a x (T R b y (T R c z d)) = T R (T B a x b) y (T B c z d)
balance c l a r = T c l a r
```

Luego de insertar un nuevo nodo rojo hay a lo sumo una única violación del invariante 1, que ocurre cuando el padre es rojo. Por lo tanto, la violación siempre ocurre en un camino $B - R - R$.

La función **balance** va arreglando y propagando hacia arriba esta violación. La (única) violación, puede aparecer en cuatro configuraciones. En todos los casos la solución es la misma, reescribir el nodo como un padre rojo con dos hijos negro.

Caso 1: raíz z negra, subárbol izquierdo y rojo y subárbol izquierdo x de este último también rojo.

```
balance B (T R (T R a x b) y c) z d = T R (T B a x b) y (T B c z d)
```



Caso 2: raíz z negra, subárbol izquierdo x rojo y subárbol derecho y de este último también rojo.

`balance B (T R a x (T R b y c) ) z d = T R (T B a x b) y (T B c z d)`



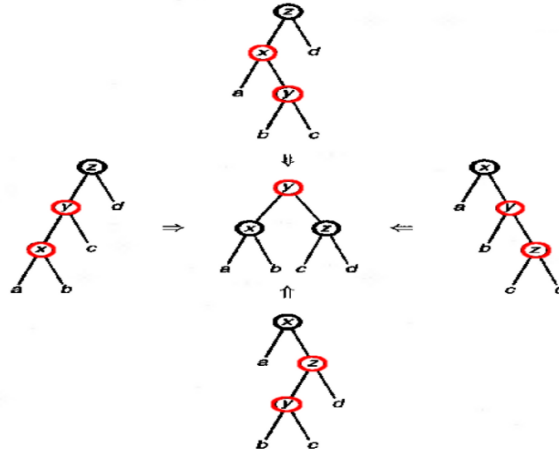
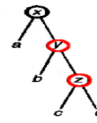
Caso 3: raíz x negra, subárbol derecho z rojo y subárbol izquierdo y de este último también rojo.

`balance B a x (T R (T R b y c) z d) = T R (T B a x b) y (T B c z d)`



Caso 4: raíz x negra, subárbol derecho y rojo y subárbol derecho z de este último también rojo.

`balance B a x (T R b y (T R c z d)) = T R (T B a x b) y (T B c z d)`



Vemos que $W_{\text{balance}} \in O(1)$, luego $W_{\text{insert}} \in O(\lg n)$.

5.2.3. Heaps

Los **heaps** (montículos) son árboles que permiten un acceso eficiente al mínimo elemento del árbol, al mantener la invariante de que todo nodo es menor a todos los demás valores de sus nodos hijos. Por lo tanto el mínimo siempre está en la raíz.

Hay varias versiones de heaps con distintos invariantes además de las tradicionales, Leftist, Binomial, Splay y Pairing Heaps. en cualquier caso, la idea es que todas estas deben tener operaciones `insert`, `findMin` y `deleteMin` eficientes, y algunas variantes también soportan eficientemente la unión de dos heaps, el `merge`.

Leftist heaps.

Un **Leftist heap** es una invariante de la estructura Heap que resulta fácil de implementar en forma inmutable. Se definen dos términos para trabajar con estas estructuras.

- **Espina derecha:** representa el camino simple desde la raíz hasta el nodo vacío más a la derecha.
- **Rango:** longitud de la espina derecha.

La invariante de la estructura Leftist heap es que el rango de cualquier hijo izquierdo es mayor o igual que el rango de su hermano de la derecha. Como consecuencia de esta y la invariante de Heap, tenemos las siguientes propiedades:

1. La espina derecha es el camino más corto a un nodo vacío.
2. La longitud de la espina derecha es a lo sumo $\lg(n + 1)$.
3. Los elementos de la espina derecha están ordenados.

La última propiedad es importante. La idea es entender al rango no solo como la longitud de la espina derecha, sino también como la longitud del camino más corto a la raíz de una hoja. Si no fuese así, los hijos podrían intercambiarse de lugar para que dicho camino quede en la espina derecha, obteniendo un menor rango, lo cual sería contradictorio. Surge luego que si el camino más corto desde la raíz hasta una hoja es r , entonces el árbol es completo hasta la profundidad r (de no serlo se encontraría un camino más corto). Ser completo hasta la profundidad r implica $n \geq 2^r - 1 \Rightarrow \lg(n + 1) \geq r$ y como $r \in \mathbb{N}$ tenemos $r \leq \lfloor \lg(n + 1) \rfloor$.

La definición de Leftist heap es la siguiente:

```
type Rank = Int
data Heap a = E | N Rank a (Heap a) (Heap a)
```

Implementación de Leftist Heaps.

```
type Rank = Int
data Heap a = E | N Rank a (Heap a) (Heap a)

-- Función que devuelve el rango de un heap.
rank :: Heap a → Rank
rank E = 0
rank (N r _ _) = r

-- Función que mergea dos heaps  $O(\lg n)$  (es la operación más importante de heaps).
-- Fusionamos siempre con el subárbol derecho porque en leftist heaps el derecho es
-- más liviano.
merge :: Ord a ⇒ Heap a → Heap a → Heap a
merge h1 E = h1
merge E h2 = h2
```

```

merge h1@(N _ x a1 b1) h2@(N _ y a2 b2) =
  if x <= y then makeH x a1 (merge b1 h2)
    else makeH y a2 (merge h1 b2)
  where
    -- Después de fusionar, nos aseguramos de que el hijo izquierdo (a) tenga
    -- mayor rank que el derecho (b).
    makeH x a b = if rank a >= rank b
      then N (rank b + 1) x a b
      else N (rank a + 1) x b a

-- Función para insertar un elemento en un heap  $O(\lg n)$ .
insert :: Ord a => a -> Heap a -> Heap a
insert x h = merge (N 1 x E E) h

-- Función que devuelve el mínimo de un heap  $O(1)$ .
findMin :: Ord a => Heap a -> a
findMin (N _ x a b) = x

-- Función que borra el elemento mínimo de un heap  $O(\lg n)$ .
deleteMin :: Ord a => Heap a -> Heap a
deleteMin (N _ x a b) = merge a b

```

La idea general del `merge` es mezclar las ramas derechas de los dos heaps según corresponda, esto es, ver quien tiene el menor dato en la raíz, si es el primero, entonces mezclamos su hijo derecho con el segundo heap, y sino mezclamos el hijo derecho del segundo heap con el primer heap. Una vez hecho esto, la función auxiliar `makeH` se encarga de colocar los dos hijos correspondientes según quien tenga el menor rango. Tenemos de aquí que $W_{\text{makeH}}, W_{\text{rank}} \in O(1)$, y junto con la propiedad de que el rango es a lo sumo logarítmico, se deduce que $W_{\text{merge}} \in O(\lg n)$, donde n es el máximo entre la cantidad de nodos de ambos heaps.

Con esta función, las demás tienen una definición sencilla, y es fácil ver que $W_{\text{insert}}, W_{\text{deleteMin}} \in O(\lg n)$, $W_{\text{findMin}} \in O(1)$.

Los leftist heap no son árboles balanceados. Lo importante de esta estructura es que siempre conocemos el camino más corto a la raíz y de esta manera usar este camino para hacer la recursión `merge` lo más corta posible.

6. Tipos Abstractos de Datos (TADs)

6.1. TADs

Los **tipos abstractos de datos (TADs)** son una forma de estructurar y organizar datos en programación. Definen un conjunto de valores y un conjunto de operaciones que pueden realizarse con esos valores, pero ocultan los detalles internos de como se implementan esos valores y operaciones.

Esto permite que los usuarios utilicen los tipos de datos sin necesidad de conocer los detalles de su implementación, lo que promueve la modularidad y abstracción en el diseño de programas.

- La idea de un tipo de datos es abstraer los detalles de implementación.
- El usuario es alguien que simplemente usa la abstracción.
- El implementador provee una implementación que se ajusta al comportamiento esperado.
- El usuario solo puede suponer el comportamiento descrito.
- Podemos ser más precisos sobre el comportamiento mediante una **especificación**.

Un TAD consiste de:

- **Un nombre de tipo.**
- **Operaciones.**
- **Especificación de comportamiento.**
 - Especificación algebraica: se describen operaciones y ecuaciones entre operaciones.
 - Modelos: se describen operaciones y como se interpretan en un modelo matemático.

6.1.1. Ejemplo - TAD de colas

Una **cola** es una estructura a la cual:

- Podemos agregar elementos.
- Podemos obtener el primer elemento.
- Podemos quitar el primer elemento.
- Podemos preguntar si está vacía.

- Existe una relación entre el orden en que se agregan elementos y se sacan (FIFO).

Esta descripción es abstracta porque refleja el comportamiento y no la implementación.

Tipo abstracto de datos para colas.

- **Nombre de tipo:** Cola.
- **Operaciones:**

```
tad Cola (A:Set) where
  import Bool
  vacia : Cola A
  poner : A → Cola A → Cola A
  primero : Cola A → A
  sacar : Cola A → Cola A
  esVacia : Cola A → Bool
```

- **Especificación algebraica:**

```
esVacia vacia = True
esVacia (poner x q) = False
primero(poner x vacia) = x
primero(poner x (poner y q)) = primero(poner y q)
sacar(poner x vacia) = vacia
sacar(poner x (poner y q)) = poner x (sacar (poner y q))
```

6.1.2. Especificaciones algebraicas de un TAD

Las **especificaciones algebraicas** deben ser mediante ecuaciones. Solo deben aparecer operaciones de TAD y variables libres (se suponen cuantificadas universalmente).

Por ejemplo, la manera correcta de escribir:

$$\text{size } x = \begin{cases} 0 & \text{si esVacia } x \\ 1 + \text{size}(\text{sacar } x) & \text{en otro caso} \end{cases}$$

sería la siguiente forma:

$$\text{size } x = \text{if } (\text{esVacia } x) \text{ then } 0 \text{ else } 1 + \text{size}(\text{sacar } x)$$

donde **0**, **1** son operadores nularios, y **+** es un operador binario de un TAD de Naturales. Además, **if then else** es un operador ternario de un TAD de Booleanos.

En algunos casos puede quedar comportamientos sin definir. Por ejemplo, la especificación no dice nada sobre **primero vacia**.

Observación.

Las especificaciones algebraicas se escriben en pseudocódigo. Muchas veces estas son sintácticamente idénticas a una posible implementación, pero no se debe confundirlas ya que son esencialmente distintas. El $=$ en implementaciones representa una definición, pero en las especificaciones algebraicas representa una igualdad.

6.2. Modelos en un TAD

La **especificación por modelos** de un TAD consiste en darle una interpretación semántica mediante un modelo al tipo de dato. La idea es dar una interpretación a cada operación de la interfaz, y una asignación a cada instancia del tipo de dato con un elemento particular del elemento del universo. Esto equivale a decir:

- Si x es una instancia del tipo, $\llbracket x \rrbracket$ es un elemento en el universo del modelo.
- Si op es una función del tipo, entonces en el modelo se tiene una función $\llbracket op \rrbracket$.

Y diremos que una implementación se ajusta a un modelo su para todo x y op :

$$\llbracket op \ x \rrbracket = \llbracket op \rrbracket \llbracket x \rrbracket$$

Como modelos de colas tomamos la secuencia $\langle x_1, x_2, \dots, x_n \rangle$ y para cada operación damos una función equivalente sobre este modelo:

vacia = $\langle \rangle$
poner x $\langle x_1, x_2, \dots, x_n \rangle = \langle x, x_1, x_2, \dots, x_n \rangle$
sacar $\langle x_1, x_2, \dots, x_n \rangle = x_n$
esVacia $\langle x_1, x_2, \dots, x_n \rangle = \text{True}$ si $n = 0$
esVacia $\langle x_1, x_2, \dots, x_n \rangle = \text{False}$ en otro caso

6.2.1. Implementaciones y costos de un TAD

Cada TAD admite diferentes implementaciones. Por ejemplo, para colas, existen dos implementaciones distintas con listas:

- Primer elemento de la cola al principio de la lista: se agregan elementos al final y se sacan del principio.

```
type Cola a = [a]
-- Wvacia = 0(1)
```

```

vacía :: Cola a
vacía = []

-- Wponer = O(n)
poner :: a → Cola a → Cola a
poner x [] = [x]
poner x (y:ys) = x : (poner y ys)

-- Wprimero = O(1)
primero :: Cola a → a
primero (x:xs) = x

-- Wsacar = O(1)
sacar :: Cola a → Cola a
sacar (x:xs) = xs

-- WesVacía = O(1)
esVacía :: Cola a → Bool
esVacía xs = null xs

```

- Primer elemento de la cola al final de lista: se agregan elementos al principio y se sacan del final.

```

type Cola a = [a]

-- Wvacía = O(1)
vacía :: Cola a
vacía = []

-- Wponer = O(1)
poner :: a → Cola a → Cola a
poner x xs = x:xs

-- Wprimero = O(n)
primero :: Cola a → a
primero (x:xs) = last (x:xs)

-- Wsacar = O(n)
sacar :: Cola a → Cola a
sacar [x] = vacía
sacar (x:xs) = x : (sacar xs)

-- WesVacía = (1)
esVacía :: Cola a → Bool
esVacía xs = null xs

```

- Implementación de colas eficiente: las implementaciones anteriores son lineales en algunas operaciones, por eso damos otra implementación más eficiente de colas. Implementamos colas usando un par de listas (*xs*, *ys*) tal que los elementos en orden sean:

$$xs ++ reverse\ ys$$

Y la implementación seguirá la siguiente invariante: si *xs* es vacía, entonces *ys* también.

Por ejemplo, la cola $\langle 1, 2, 3, 4, 5 \rangle$ podría ser $([1, 2], [5, 4, 3])$ o $([1], [5, 4, 3, 2])$.

```

type Cola a = ([a], [a])

-- Wvacía = O(1)
vacía :: Cola a
vacía = ([], [])

-- Wponer = O(1)
poner :: a → Cola a → Cola a
poner x (ys, zs) = validar (ys, x:zs)

-- Wprimero = O(1)
primero :: Cola a → a
primero (x:xs, ys) = x

-- Wsacar = O(|ys|), O(1) (amortizado)
sacar :: Cola a → Cola a
sacar (x:xs, ys) = validar (xs, ys)

-- WesVacía = O(1)
esVacía :: Cola a → Bool
esVacía (xs, ys) = null xs

validar :: Cola a → Cola a
validar (xs, ys) = if null xs then (reverse ys, []) else (xs, ys)

```

El costo amortizado de una operación es una medida promedio del costo de realizar una secuencia de operaciones en una estructura de datos durante un período de tiempo.

En la función `validar`, la llamada a `reverse` es $O(n)$ pero se hace cada tanto, solo en algunos pocos casos. Como en todos los otros casos de llamada a `validar` son $O(1)$, nos estamos ahorrando costo para cuando tengamos que aplicar `reverse`. Es decir, la operación `sacar` termina siendo $O(1)$, pues hemos acumulado tantos $O(1)$ (como monedas) al momento de costear una operación $O(n)$ que es `reverse`.

6.2.2. TADs en Haskell

Una forma de implementar un **TAD en Haskell** es mediante clase de tipos:

```

class Cola t where
  vacía :: t
  poner :: a → t a → t a
  sacar :: t a → t a
  primero :: t a → a
  esVacía :: t a → Bool

```

Y la implementación de un TAD es una instancia:

```

instance Cola [] where
  vacía = []
  poner x xs = x:xs
  sacar xs = init xs
  primero = last
  esVacía xs = null xs

```

Usamos un TAD con una función polimórfica en el TAD. Notar que la siguiente función `ciclar` funciona para cualquier instancia de la Cola. O lo que es lo mismo, funciona para cualquier implementación del TAD:

```
ciclar :: Cola t → Int → t a → t a
ciclar 0 cola = cola
ciclar n cola = ciclar (n-1) (poner (primero cola) (sacar cola))
```

6.2.3. Conclusión

- **Especificación:** que operaciones tiene el TAD y como se comportan. Es única.
- **Implementación:** como se realizan las operaciones y cuanto cuestan. Puede haber varias implementaciones (con diferentes costos). Todas deben garantizar el comportamiento dado por la especificación.
- **Uso:** solo puede suponer el comportamiento dado por la especificación. Se elige implementación de acuerdo al uso (menor costo para determinado uso).
- Los TADs ocultan detalles de implementación, el comportamiento se describe algebraicamente o proveyendo un modelo y cada implementación debe tener una especificación de costo.

6.2.4. TADs básicos

TAD de Booleanos.

```
tad Bool where
true  : Bool
false : Bool
not   : Bool → Bool
and   : Bool → Bool → Bool
or    : Bool → Bool → Bool
(⇒)  : Bool → Bool → Bool
if_then_else : Bool → E → E → E
```

```
not true = false
not false = true
and true x = x
and false = false
or true x = true
or false x = x
true ⇒ x = x
false ⇒ x = true
if true then x else y = x
if false then x else y = y
```

TAD de Naturales.

```
tad Nat where
  import Bool
  0 : Nat
  succ : Nat → Nat
  (>) : Nat → Nat → Bool
  (==) : Nat → Nat → Bool
  (+) : Nat → Nat → Nat
  (-) : Nat → Nat → Nat
```

$$\begin{aligned}
x + 0 &= x \\
x + (\text{succ } y) &= \text{succ}(x + y) \\
0 - x &= 0 \\
x - 0 &= x \\
(\text{succ } x) - (\text{succ } y) &= x - y \\
0 == 0 &= \text{true} \\
(\text{succ } x) == 0 &= \text{false} \\
0 == (\text{succ } y) &= \text{false} \\
0 > x &= \text{false} \\
(\text{succ } x) > 0 &= \text{true} \\
(\text{succ } x) > (\text{succ } y) &= x > y
\end{aligned}$$

TAD de Int.

```
tad Int where
  import Bool
  0 : Int
  succ : Int → Int
  pred : Int → Int
  neg : Int → Int
  abs : Int → Int
  (+) : Int → Int → Int
  (-) : Int → Int → Int
  (*) : Int → Int → Int
  div : Int → Int → Int
  mod : Int → Int → Int
  min : Int → Int → Int
  max : Int → Int → Int
  (>) : Int → Int → Bool
  (==) : Int → Int → Bool
```

Se considera la especificación usual para este tipo.

TAD de Tupla.

```
tad Tupla_n (A1, ..., An : Set) where
  p1 : Tupla_n (A1, ..., An) → A1
  ...
  pn : Tupla_n (A1, ..., An) → An
  (_, ..., _) : A1 → ... → An → Tupla_n (A1, ..., An)
```

$$\begin{aligned}
p_1(x_1, \dots, x_n) &= x_1 \\
&\vdots \\
p_n(x_1, \dots, x_n) &= x_n
\end{aligned}$$

TAD de Maybe

```
tad Maybe (A:Set) where
  import Bool
  Nothing : Maybe A
  just : A → Maybe A
  isNothing : Maybe A → Bool
  fromJust : Maybe A → A
```

$$\begin{aligned}
\text{isNothing } (\text{Nothing}) &= \text{true} \\
\text{isNothing } (\text{Just } x) &= \text{false} \\
\text{fromJust } (\text{Just } x) &= x
\end{aligned}$$

6.3. Verificación de especificaciones

Los TADs presentan una entidad con ciertas operaciones y con las especificaciones se establecen las reglas que tales operaciones cumplen. Dada una implementación, debemos saber si esta cumple con las especificaciones.

Dada una implementación de un TAD en Haskell, el sistema de tipos asegura que los tipos de las operaciones sean correctos, pero la verificación de la especificación la debe realizar el programador.

6.3.1. Razonamiento ecuacional

Haskell permita **razonar ecuacionalmente** acerca de las definiciones en forma similar al álgebra. Por ejemplo, si tenemos la siguiente función:

```
reverse :: [a] → [a]
reverse [] = []
reverse (x:xs) = reverse xs ++ [x]
```

Podemos probar la propiedad: `reverse [x] = [x]`

<code>= reverse [x]</code>	$\langle \text{def. listas} \rangle$
<code>= reverse (x : [])</code>	$\langle \text{reverse 2da def} \rangle$
<code>= reverse [] ++ [x]</code>	$\langle \text{reverse 1ra def} \rangle$
<code>= [] ++ [x]</code>	$\langle ++ \text{ 1ra def} \rangle$
<code>= [x]</code>	

6.3.2. Patrones disjuntos

Al verificar la correctitud de un programa, es fundamental garantizar que su especificación sea clara y libre de ambigüedades. Una técnica clave para lograr esto es el uso de **patrones disjuntos** en definiciones por casos, ya que simplifican el razonamiento ecuacional y evitan solapamientos no deseados

Por ejemplo, consideremos la siguiente función `esCero`, que verifica si un entero es cero:

```
esCero :: Int → Bool
esCero 0 = True
esCero n = False
```

Aunque esta definición es correcta, introduce una dependencia implícita del orden de las ecuaciones, el segundo patrón (n) cubre todos los enteros, incluyendo el 0, pero solo se evalúa si el primer patrón no coincide.

Esto obliga al verificador a recordar que el orden importa, lo que puede complicar el razonamiento formal.

Una alternativa más robusta es restringir el segundo caso para que solo aplique cuando n no sea cero, haciendo los patrones disjuntos.

```
esCero~ :: Int → Bool
esCero~ 0 = True
esCero~ n | n /= 0 = False
```

Aquí los patrones no se solapan, por lo que el verificador no necesita considerar prioridades. Cada ecuación cubre un conjunto de valores mutuamente excluyentes.

6.3.3. Extensionalidad

El **principio de extensionalidad** nos da un criterio claro para determinar cuándo dos funciones son iguales, centrándose únicamente en su comportamiento observable y no en sus detalles internos.

Dados dos funciones $f, g :: A \rightarrow B$, decimos que f y g son iguales, si para todo valor de entrada x en el dominio A se cumple $f(x) = g(x)$. Es decir, no importa como estén implementadas f y g (código y eficiencia) sino que produzcan el mismo resultado para cada entrada posible. Por ejemplo, los algoritmos de ordenamiento son iguales, pues para una misma entrada (lista) devuelven la misma salida (lista ordenada).

$$f = g \Leftrightarrow \forall x :: A. f\ x = g\ x$$

De esta manera, podemos hacer análisis por casos sobre los elementos posibles del dominio A de la función.

Por ejemplo, probamos la siguiente propiedad utilizando los posibles casos de x , que es una variable booleana y cuyos valores son únicamente **True** y **False**:

$$\text{not } (\text{not } x) = x$$

```
not :: Bool → Bool
not False = True
not True = False
```

Caso $x = \text{False}$.

$$\begin{aligned} &= \text{not } (\text{not } \text{False}) \\ &= \text{not } \text{True} && \langle \text{not def 1} \rangle \\ &= \text{False} && \langle \text{not def 2} \rangle \end{aligned}$$

Caso $x = \text{True}$.

$$\begin{aligned} &= \text{not } (\text{not } \text{True}) \\ &= \text{not } \text{False} && \langle \text{not def 2} \rangle \\ &= \text{True} && \langle \text{not def 1} \rangle \end{aligned}$$

6.3.4. Inducción

Para poder probar propiedades acerca de programas recursivos usualmente necesitamos usar **inducción**. La inducción nos da una forma de escribir una prueba infinita de una manera finita.

1. Inducción sobre los naturales.

Para probar $P(n)$ para todo $n \in \mathbb{N}$, probamos $P(0)$ y probamos que para cualquier m , si $P(m)$ entonces vale $P(m + 1)$.

- La prueba $P(0)$ es el **caso base**.
- La prueba de que $P(m) \rightarrow P(m + 1)$ es el **paso inductivo**.
- El suponer $P(m)$ verdadero es la **hipótesis de inducción**.

2. Inducción fuerte sobre los naturales.

Para probar $P(n)$ para todo $n \in \mathbb{N}$, probamos que para cualquier m , si vale $P(i)$ para todo $i < m$, entonces vale $P(m)$.

- No hay caso base.
- Suponemos verdadero $P(i)$ para todo $i < m$ (hipótesis de inducción).

3. Inducción sobre otros conjuntos.

Podemos usar la inducción sobre los naturales para obtener inducción sobre otros conjuntos. Por ejemplo, podemos hacer inducción sobre la altura de un árbol o la longitud de una lista.

En general, dada una función $f : A \rightarrow \mathbb{N}$, y una propiedad P sobre elementos de A , podemos definir:

$$Q(n) = \forall a :: A. f\ a = n \Rightarrow P(a)$$

y así transformamos una propiedad sobre A en una sobre $\mathbb{N}$.

4. Inducción estructural.

Dada una propiedad P sobre un tipo de datos algebraico T , para probar que vale $P(t) \forall t :: T$:

- Probamos $P(t)$ para todo t dado por un constructor no recursivo.
- Para todo t dado por un constructor con instancias recursivas $t_1, \dots, t_k$, probamos que si vale $P(t_i)$ para cada i , entonces vale $P(t)$.

6.3.5. Ejemplos

▪ Ejemplo inducción sobre otros conjuntos.

Sea T un árbol binario definido como sigue

```
data Bin = Null | Leaf | Node Bin Bin

cantleaf :: Bin → Int
cantleaf Null = 0
cantleaf Leaf = 1
cantleaf (Node t u) = cantleaf t + cantleaf u

cantnode :: Bin → Int
cantnode (Node t u) = 1 + cantnode t + cantnode u
cantnode _ = 0

height :: Bin → Int
height (Node t u) = 1 + max (height t) (height u)
height _ = 0
```

Queremos probar la siguiente propiedad

$$\forall t :: Bin : \text{cantleaf } t \leq \text{cantnode } t + 1$$

Prueba. Definimos la siguiente propiedad a probar:

$$Q(n) = \text{height}(t) = n \Rightarrow \text{cantleaf } t \leq \text{cantnode } t + 1 \quad \forall t :: Bin$$

Usamos la segunda forma de inducción (fuerte) y suponemos por hipótesis inductiva:

$$\forall i < n, \text{ si } \text{height}(t) = i, \text{ entonces } \text{cantleaf } t \leq \text{cantnode } t + 1$$

Ahora hacemos un análisis por casos de n :

- Si $n = 0$, entonces la hipótesis inductiva no se aplica y debemos probar directamente.

Si $\text{height}(t) = 0$, entonces por definición de **height**, t tiene una hoja o está vacío:

- Si tiene una hoja: $\text{cantleaf } t = 1 \leq \text{cantnode } t + 1 = 0 + 1 = 1$.
- Si está vacío: $\text{cantleaf } t = 0 \leq \text{cantnode } t + 1 = 0 + 1 = 1$

- Si $n > 0$ y $\text{height}(n) = n$, entonces podemos calcular:

$$\begin{aligned} &= \text{cantleaf } t \\ &= \text{cantleaf } (\text{Node } u \text{ } v) && \langle \text{height}(t) > 0 \rangle \\ &= \text{cantleaf } u + \text{cantleaf } v && \langle \text{cantleaf def 3} \rangle \\ &\leq \text{cantleaf } u + 1 + \text{cantleaf } v + 1 && \langle \text{HI } (\text{height } u < n) \text{ y } (\text{height } v < n) \rangle \end{aligned}$$

$$\begin{aligned}
&= (1 + \text{cantnode } u + \text{cantnode } v) + 1 && \langle \text{asociatividad} \rangle \\
&= \text{cantnode } (\text{Node } u \ v) + 1 && \langle \text{cantnode def 1} \rangle \\
&= \text{cantnode } t + 1
\end{aligned}$$

Y luego por inducción hemos probado la propiedad Q . Así, hemos podido probar una propiedad sobre árboles usando inducción sobre los naturales.

Sin embargo, es más práctico hacer la inducción sobre la estructura con inducción estructural.

■ **Ejemplo inducción estructural.**

Sea T el árbol binario definido en el anterior ejemplo, probaremos la misma propiedad que antes pero utilizando inducción estructural:

$$\forall t :: \text{Bin} : \text{cantleaf } t \leq \text{cantnode } t + 1$$

- **Caso Null:** $\text{cantleaf } \text{Null} = 0 \leq 1 = 0 + 1 = \text{cantnode } \text{Null} + 1$.
- **Caso Leaf:** $\text{cantleaf } \text{Leaf} = 1 \leq 1 = 0 + 1 = \text{cantnode } \text{Leaf} + 1$.
- **Caso Node $u \ v$:** utilizaremos las siguientes hipótesis inductivas:

$$\begin{aligned}
&\text{cantleaf } u \leq \text{cantnode } u + 1 \\
&\text{cantleaf } v \leq \text{cantnode } v + 1
\end{aligned}$$

$$\begin{aligned}
&= \text{cantleaf } (\text{Node } u \ v) \\
&= \text{cantleaf } u + \text{cantleaf } v && \langle \text{cantleaf def 3} \rangle \\
&\leq \text{cantnode } u + 1 + \text{cantnode } v + 1 && \langle \text{HI } (\text{height } u < n) \text{ y } (\text{height } v < n) \rangle \\
&= (1 + \text{cantnode } u + \text{cantnode } v) + 1 && \langle \text{asociatividad} \rangle \\
&= \text{cantnode } (\text{Node } u \ v) + 1 && \langle \text{cantnode def 1} \rangle \\
&= \text{cantnode } t + 1
\end{aligned}$$

■ **Inducción estructural para listas.**

Dada una propiedad P sobre listas, para probar $\forall xs :: [a]. P(xs)$:

- Probamos $P([])$.
- Probamos que si $P(xs)$ entonces vale $P(x : xs)$.

Por ejemplo, probamos la siguiente propiedad:

$$\text{reverse } xs ++ ys = \text{reverse } ys ++ xs$$

```
reverse :: [a] → [a]
reverse [] = []
reverse (x:xs) = reverse xs ++ [x]
```

- **Caso xs = []:**

$$\begin{aligned}
&= \text{reverse } ([] ++ \text{ys}) \\
&= \text{reverse ys} && \langle ++ \text{ def 1} \rangle \\
&= \text{reverse ys} ++ [] && \langle ++ \text{ def 1} \rangle \\
&= \text{reverse ys} ++ \text{reverse } [] && \langle \text{reverse def 1} \rangle
\end{aligned}$$

- **Caso xs.** Suponemos que $P(xs)$ es verdadero y probaremos para $P(x:xs)$.

$$\begin{aligned}
&= \text{reverse } (x:xs ++ \text{ys}) \\
&= \text{reverse } (xs ++ \text{ys}) ++ [x] && \langle \text{reverse def 2} \rangle \\
&= \text{reverse ys} ++ \text{reverse xs} ++ [x] && \langle \text{HI} \rangle \\
&= \text{reverse ys} ++ \text{reverse } (x:xs) && \langle \text{reverse def 2} \rangle
\end{aligned}$$

■ Ejemplo compilador correcto.

Dado un lenguaje aritmético simple, cuyo AST se ve expresado en la definición `data` y su semántica denotacional está dada por el evaluador, queremos compilar el lenguaje a la máquina de stack.

```
data Expr = Val Int | Add Expr Expr

-- Evaluador.
eval :: Expr → Int
eval (Val n) = n
eval (Add x y) = eval x + eval y

-- Máquina de stack.
type Stack = [Int]
type Code = [Op]
data Op = PUSH Int | ADD

-- Dado un código y un stack, devuelve el stack luego de procesar el código.
exec :: Code → Stack → Stack
exec [] s = s
exec (PUSH n:c) s = exec c (n:s)
exec (ADD:c) (m:n:s) = exec c (n+m:s)

-- Compilador: traduce una expresión Expr a Code que puede ejecutar la máquina de pila.
comp :: Expr → Code
comp (Val n) = [PUSH n]
comp (Add x y) = comp x ++ comp y ++ [ADD]

-- Ejemplo.
> e = Add (Add (Val 2) (Val 3)) (Val 4) -- :: Expr
> eval e = 2 + 3 + 4 = 9 -- :: Int
```

```

> comp e = comp (Add (Val 2) (Val 3)) ++ comp (Val 4) ++ [ADD]
          = (comp (Val 2) ++ comp (Val 3) ++ [ADD]) ++ [PUSH 4] ++ [ADD]
          = [PUSH 2] ++ [PUSH 3] ++ [ADD] ++ [PUSH 4] ++ [ADD]
          = [PUSH 2, PUSH 3, ADD, PUSH 4, ADD]

> exec (comp e) [] = exec [PUSH 2, PUSH 3, ADD, PUSH 4, ADD] []
                  = exec [PUSH 3, ADD, PUSH 4, ADD] [2]
                  = exec [ADD, PUSH 4, ADD] [3,2]
                  = exec [PUSH 4, ADD] [5]
                  = exec [ADD] [4,5]
                  = exec [] [9]
                  = [9]

```

El compilador será correcto si se cumple la siguiente propiedad:

$$\text{exec}(\text{comp } e) [] = [\text{eval } e] \forall e$$

[PRUEBA VER SLIDES 23 a 29 en 'RAZONANDO CON PROGRAMAS']

7. Estructuras paralelizables

Si recordamos la implementación del algoritmo `msort` sobre listas, se llega a que las cotas de su trabajo y profundidad eran $W_{\text{msort}}(n) \in O(n \lg n)$ y $S_{\text{msort}}(n) \in O(n)$. La definición de paralelismo para este algoritmo nos da $P = \frac{W_{\text{msort}}}{S_{\text{msort}}} = \frac{n \lg n}{n} = \lg n$, lo cual dice que muy pocos procesadores pueden usarse de forma eficiente. El problema radica en que el trabajo no puede ser mejorado debido a que esta versión del `msort` trabaja sobre listas. Esto debido a que son estructuras inherentes y no paralelas. Su modelo de construcción es secuencial, nodo tras nodo hasta llegar al final. La idea para mejorar la profundidad de la implementación de mergesort y cualquier algoritmo en general será buscar una estructura de datos que admita altos niveles de paralelización.

7.1. Implementación paralela del mergesort

7.1.1. Recordando mergesort

El algoritmo de ordenación `mergesort` es un clásico ejemplo de Divide & Conquer, dividimos la entrada en dos (`split`), ordenamos recursivamente y juntamos las dos mitades ordenadas (`merge`).

Pseudocódigo del algoritmo.

```
msort : [Int] → [Int]
msort [] = []
msort [x] = [x]
msort xs = let
    (ls, rs) = split xs
    (ls', rs') = (msort ls || msort rs)
in
    merge(ls', rs')

split : [Int] → ([Int], [Int])
split [] = []
split [x] = ([x], [])
split (x:y:zs) = let
    (xs, ys) = split zs
in
```

(x:xs,y:ys)

```

merge : ([Int], [Int]) → [Int]
merge([], ys) = ys
merge(xs, []) = xs
merge(x:xs,y:ys) = if x <= y
                    then x:merge(xs,y:ys)
                    else y:merge(x:xs,ys)

```

Trabajo del algoritmo.

- $W_{\text{split}}(n) \in O(n)$.
- $W_{\text{merge}}(n) \in O(n)$.
- $W_{\text{msort}}(n) = c_0n + 2W_{\text{msort}}\left(\frac{n}{2}\right) + c_1n + c_2$.

Por lo tanto, $W_{\text{msort}}(n) \in O(n \lg n)$.

Profundidades del algoritmo.

- $S_{\text{split}}(n) \in O(n)$.
- $S_{\text{merge}}(n) \in O(n)$.
- $S_{\text{msort}}(n) = k_0n + S_{\text{msort}}\left(\frac{n}{2}\right) + k_1n + k_2$.

Por lo tanto, $W_{\text{msort}}(n) \in O(n)$.

7.1.2. Paralelizando mergesort

Como dijimos, la elección de la estructura de datos influye en la profundidad del algoritmo. En lugar de listas trabajaremos con el siguiente tipo de datos:

data BT a = Empty | Node (BT a) a (BT a)

Paralización más efectiva: debido a la estructura jerárquica de los árboles binarios de búsqueda, es más fácil dividir y procesar los datos de manera paralela. Cada subárbol puede ser procesado de forma independiente, lo que facilita la paralelización del algoritmo y puede llevar a una mejor utilización de recursos de hardware en entornos paralelos.

7.1.3. Mergesort sobre árboles

Previo a las definiciones de las funciones recordamos la definición de un **árbol binario de búsqueda**. Un elemento de un árbol binario de búsqueda está ordenado sii:

1. Es el árbol `Empty`.
2. Es un `(Node l x r)` y además:
 - `l` está ordenado.
 - `r` está ordenado.
 - todos los elementos en `l` son $\leq x$.
 - todos los elementos en `r` son $> x$.

Además, un árbol ordenado induce una lista ordenada (recorrido inorder):

```
-- VERSION O(n)
listar :: BT a -> a
listar t = listarAux t []
  where
    listarAux :: BT a -> [a] -> [a]
    listarAux Empty xs = xs
    listarAux (Node l x r) xs = listarAux l (x:listarAux r xs)
```

Split en mergesort.

Tenemos que $W_{\text{split}} = O(1)$ y $S_{\text{split}} = O(1)$, pues el árbol ya está dividido en sus subárboles izquierdo y derecho.

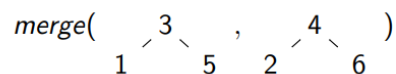
Msort en mergesort.

```
msort :: BT a -> BT a
msort Empty = Empty
msort (Node l x r) = let (l~, r~) = msort l || msort r
  in merge (merge l~ r~) (Node Empty x Empty)
```

Merge en mergesort.

Queremos definir la función `merge` tal que $W_{\text{merge}} = O(n)$ y S_{merge} sea mejor que $O(n)$. Para eso:

1. Consideremos el siguiente caso:



2. Elegimos guiarnos por el primer argumento y utilizar su raíz en el nuevo árbol mergeado:

$$\text{merge}\left(\begin{array}{c} 3 \\ 1 \quad 5 \end{array}, \begin{array}{c} 4 \\ 2 \quad 6 \end{array}\right) = \begin{array}{c} 3 \\ \text{merge}(?, ?) \quad \text{merge}(?, ?) \end{array}$$

$$\text{merge}\left(\begin{array}{c} 3 \\ 1 \quad 5 \end{array}, \begin{array}{c} 4 \\ 2 \quad 6 \end{array}\right) = \begin{array}{c} 3 \\ \text{merge}(1, ?) \quad \text{merge}(5, ?) \end{array}$$

3. El 1 segura va a la izquierda y el 5 a la derecha:
4. Separamos el segundo argumento en árboles menores a 3 y mayores a 3:

$$\text{merge}\left(\begin{array}{c} 3 \\ 1 \quad 5 \end{array}, \begin{array}{c} 4 \\ 2 \quad 6 \end{array}\right) = \begin{array}{c} 3 \\ \text{merge}(1, 2) \quad \text{merge}(5, 4) \\ \quad \quad \quad \quad \quad 6 \end{array}$$

5. Finalmente obtenemos:

Implementación merge.

```
merge :: BT Int -> BT Int -> BT Int
merge Empty t2 = t2
merge (Node l1 x r1) t2 =
    let (l2, r2) = splitAt t2 x
        (l~, r~) = merge l1 l2 || merge r1 r2
    in Node l~ x r~

splitAt :: BT Int -> Int -> BT Int X BT Int
splitAt Empty _ = (Empty, Empty)
splitAt (Node l x r) y =
    if y < x then let (l1, lr) = splitAt l y
                    in (l1, Node lr x r)
    else let (r1, rr) = splitAt r y
          in (Node l x r1, rr)
```

7.1.4. Profundidad de merge en mergesort sobre árboles

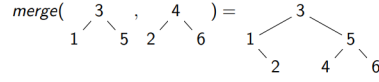
Profundidad de splitAt.

Sea h la altura del árbol, entonces $S_{\text{splitAt}}(h) = k + S_{\text{splitAt}}(h - 1)$, y por lo tanto $S_{\text{splitAt}} \in O(h)$.

Profundidad de merge.

Sean h_1 y h_2 las altura de los árboles argumento:

- $S_{\text{merge}}(h_1, h_2) \leq k + S_{\text{splitAt}}(h_2) + \max(S_{\text{merge}}(h_1 - 1, h_{21}), S_{\text{merge}}(h_1 - 1, h_{22}))$



donde h_{21} y h_{22} son las alturas de los árboles devueltos por `splitAt`.

- $S_{\text{merge}}(h_1, h_2) \leq k + S_{\text{splitAt}}(h_2) + \max(S_{\text{merge}}(h_1 - 1, h_2), S_{\text{merge}}(h_1 - 1, h_2))$

pues estamos acotando $h_{21} \leq h_2$ y $h_{22} \leq h_2$.

- $S_{\text{merge}}(h_1, h_2) \leq k'h_2 + S_{\text{merge}}(h_1 - 1, h_2)$

pues $S_{\text{splitAt}}(h_2) \in O(h_2)$. Luego, del segundo término $S_{\text{merge}}(h_1 - 1, h_2)$ resulta que estamos sumando h_1 veces el término $(k'h_2)$. Por lo tanto, $S_{\text{merge}}(h_1, h_2) \in O(h_1 \cdot h_2)$.

Si n es el tamaño del árbol, y el árbol está balanceado entonces $h \in O(\lg(n))$, y por lo tanto $S_{\text{merge}}(n) \in O(\lg(n)^2)$.

Profundidad de `msort`.

- $S_{\text{msort}}(n) \leq k + \max(S_{\text{msort}}(\frac{n}{2}), S_{\text{msort}}(\frac{n}{2})) + S_{\text{merge}}(\lg n, \lg n) + S_{\text{merge}}(2 \lg n, 1)$

donde el término $S_{\text{merge}}(2 \lg n, 1)$ proviene de `merge (merge l', r')` (`Node Empty x Empty`), y además $(\text{merge } l \ r \leq \text{altura } l + \text{altura } r)$.

- $S_{\text{msort}}(n) \leq k + S_{\text{msort}}(\frac{n}{2}) + k_1 \lg(n)^2 + k_2 \lg(n)$.

pues $S_{\text{merge}}(h_1, h_2) \in O(h_1, h_2)$ para el tercer término.

- $S_{\text{msort}}(n) \leq k + S_{\text{msort}}(\frac{n}{2}) + k_3 \lg(n)^2$

y por lo tanto, $S_{\text{msort}}(n) \in O(\lg(n)^3)$.

7.1.5. Función `rebalance`

Este análisis de la profundidad tiene un error grave. La profundidad de `merge` suponía árboles balanceados, pero en `msort` llamamos a `merge` con el resultado de las llamadas recursivas. Este problema se puede arreglar con una función **`rebalance`** encargada de rebalancear el árbol.

```
msort :: BT a -> BT a
msort Empty = Empty
msort (Node l x r) =
  let (l', r') = msort l || msort r
  in rebalance (merge (merge l' r') (Node Empty x Empty))
```

7.2. Funciones de alto orden en árboles

7.2.1. Función map, sum, flatten y reduce

Trabajamos ahora sobre otra variante de árboles binarios:

```
data T a = Empty | Leaf a | Node (T a) (T a)
```

- **Función map:** recibe una función y se la aplica a cada elemento de un árbol dado.

```
mapT :: (a → b) → T a → T b
mapT f Empty = Empty
mapT f (Leaf x) = Leaf (f x)
mapT f (Node l r) =
  let (l', r') = mapT f l || mapT f r
  in Node l' r'
```

Si suponemos que $f \in O(1)$, entonces la función map tiene $W_{\text{mapT}} = O(n)$ y $S_{\text{mapT}} = O(\lg n)$.

- **Función sum:** suma todos los elementos de un árbol de enteros.

```
sumT :: T Int → Int
sumT Empty = 0
sumT (Leaf x) = x
sumT (Node l r) =
  let (l', r') = sumT l || sumT r
  in l' ++ r'
```

Para esta función tenemos $W_{\text{sumT}} = O(n)$ y $S_{\text{sumT}} = O(\lg n)$.

- **Función flatten:** dado un árbol de cadenas, lo aplana uniendo todas sus cadenas en una sola.

```
flattenT :: T String → String
flattenT Empty = []
flattenT (Leaf xs) = xs
flattenT (Node l r) =
  let (l', r') = flattenT l || flattenT r
  in l' ++ r'
```

- **Función reduce:** dada una operación binaria f , un neutro e , y un árbol $T a$, combina recursivamente los valores del árbol usando f , devolviendo un único valor del mismo tipo.

```
reduceT :: (a → a → a) → a → T a → a
reduceT f e Empty = e
reduceT f e (Leaf x) = x
reduceT f e (Node l r) =
  let (l', r') = reduceT f e l || reduceT f e r
  in f l' r'
```

Si suponemos que $f \in O(1)$ entonces $W_{\text{reduceT}}(n) \in O(n)$ y $S_{\text{reduceT}} \in O(\lg n)$. Podemos utilizar la función **reduce** para expresar las funciones **sumT** y **flattenT**:

```
sumT = reduceT (+) 0
flattenT = reduceT (++) []
```

7.2.2. Ejemplo: contar longitud de la línea con más palabras en un texto

Queremos saber la longitud (en palabras) de la línea con más palabras en un texto:

```
lolile : String → Int
```

```
-- Recibe un string, le aplica la función words que divide una cadena en un árbol de
-- palabras (separa por espacios) y le aplica la función que convierte cada
-- palabra al número 1. Finalmente, suma todos los números 1, equivalente a la
-- cantidad de palabras que tiene un string (línea).
wordcount :: String → Int
wordcount = sumT . mapT (λ _ → 1) . word

-- Recibe un string y aplica la función lines que divide una cadena en un árbol de
-- líneas. Luego aplica la función wordcount a cada línea para contar cuantas
-- palabras tiene. Por último, con el reduce y la función de combinación max 0, se
-- queda con el máximo entre todos los conteos de palabras.
lolile = reduceT max 0 . mapT wordcount . lines

-- texto = "Hola\nEsto es una prueba\nOk"
-- lines texto → ["Hola", "Esto es una prueba", "Ok"]
-- mapT wordcount ... → [1, 4, 1]
-- reduceT max 0 ... → 4
-- lolile texto == 4
```

7.2.3. mapreduce

Hacer un `mapT` y luego un `reduceT` es ineficiente, ya que `mapT` genera un árbol que es inmediatamente consumido por `reduceT`:

- `mapT f t` recorre el árbol `t` aplicando `f` a cada hoja, generando un nuevo árbol con los valores transformados.
- `reduceT g e` vuelve a recorrer ese nuevo árbol, combinando los elementos con la función `g`.

Es decir, se recorren dos veces los árboles, y además se construye uno intermedio que no sirve más que para ser reducido. Esto es ineficiente tanto en tiempo como espacio.

Solución: mapreduce.

La idea es fusionar esas dos operaciones en una sola función que:

- Transforma cada hoja con `f` (`mapT`).
- Combina los resultados con `g` (`reduceT`).

```
mapreduce :: (a → b) → (b → b → b) → b → T a → b
mapreduce f g e = mr
  where mr Empty = e
        mr (Leaf a) = f a
        mr (Node l r) = let (l~, r~) = mr l || mr r
                          in g l~ r~
```

8. Secuencias

8.1. Definición

Seq es un TAD para representar secuencias de cualquier tipo de elementos.

Una **secuencia** s se denota como $\langle x_0, x_1, \dots, x_n \rangle$, y $|s| = n$ representa la longitud de una secuencia s .

Para i válido y secuencia s , notamos como s_i a su i -ésima proyección (elemento) y s' es una subsecuencia de s si existe secuencia I de índices válidos en s estrictamente crecientes tal que $s'_i = s_{I_i}$ (por ejemplo, en $\langle 0, 1, 2, 3, 4, 5, 6 \rangle$ la secuencia $\langle 2, 5, 6 \rangle$ es una subsecuencia de la primera).

8.2. Operaciones en secuencias

Sea s una secuencia y x un elemento, se definen las siguientes operaciones:

empty : **Seq** a

Representa la secuencia $\langle \rangle$.

singleton : $a \rightarrow \text{Seq } a$

singleton x representa la secuencia $\langle x \rangle$.

length : **Seq** $a \rightarrow \mathbb{N}$

length s evalúa a $|s|$.

nth : **Seq** $a \rightarrow \mathbb{N} \rightarrow a$

Si i es un índice válido, entonces **nth** s i evalúa a s_i .

toSeq : $[a] \rightarrow \text{Seq } a$

Dada una lista xs , **toSeq** xs , nos da la representación de xs como secuencia (respetando índices).

tabulate : $(\mathbb{N} \rightarrow a) \rightarrow \mathbb{N} \rightarrow \text{Seq } a$

tabulate f n evalúa a la secuencia $\langle f\ 0, f\ 1, f\ 2, \dots, f\ (n-1) \rangle$.

map : $(a \rightarrow b) \rightarrow \text{Seq } a \rightarrow \text{Seq } b$

`map` f s evalúa a la secuencia $\langle f\ s_0, f\ s_1, f\ s_2, \dots, f\ s_{|s|-1} \rangle$.

`filter` $(a \rightarrow \text{Bool}) \rightarrow \text{Seq } a \rightarrow \text{Seq } a$

`filter` p s evalúa a la subsecuencia más larga en la que p vale para todo sus elementos.

`append` $\text{Seq } a \rightarrow \text{Seq } a \rightarrow \text{Seq } a$

`append` s t es la secuencia $\langle s_0, s_1, \dots, s_{|s|-1}, t_0, t_1, \dots, t_{|t|-1} \rangle$.

`take`, `drop` $\text{Seq } a \rightarrow \mathbb{N} \rightarrow \text{Seq } a$

`take` s n evalúa a la secuencia $\langle s_0, s_1, s_{\min(|s|,n)-1} \rangle$.

`drop` s n evalúa a la secuencia $\langle s_n, \dots, s_{|s|-1} \rangle$.

8.3. Vista de secuencias como árbol

La siguiente operación nos permite ver a las secuencias como si fueran un árbol. La idea de descomponer la secuencia en una estructura de árbol es para facilitar ciertas operaciones. Notar que `TreeView` no es un tipo recursivo.

```
data TreeView a = EMPTY | ELT a | NODE (Seq a) (Seq a)
show : Seq a → TreeView a
```

- Si $|s| = 0$, `showt` s evalúa a `EMPTY`.
- Si $|s| = 1$, `showt` s evalúa a `ELT` s_0 .
- Si $|s| > 1$, `showt` s evalúa a `NODE` (`take` s $\frac{|s|}{2}$) (`drop` s $\frac{|s|}{2}$).

8.4. La operación foldr

La función `foldr` (**fold right**) es una operación de plegado que procesa una secuencia de elementos aplicando una función binaria de derecha a izquierda, acumulando un resultado.

$$\text{foldr} : (a \rightarrow b \rightarrow b) \rightarrow b \rightarrow \text{Seq } a \rightarrow b$$

donde:

- $(a \rightarrow b \rightarrow b)$: es una función de dos argumentos que toma un elemento de la secuencia (a) y un acumulador (b), y devuelve un nuevo acumulador (b).
- b : es el valor inicial del acumulador.

- **Seq** a : es la secuencia de elementos a procesar.
- b : es el valor resultante después de aplicar la función a todos los elementos de la secuencia (acumulador final).

Ejemplo.

foldr $(+)$ e $s = s_0 + (s_1 + (\dots (s_n + e)))$

e es el valor inicial del acumulador, en este caso, la secuencia vacía.

foldr va aplicando la función $(+)$ a todos los elementos de la secuencia, en este caso de derecha a izquierda.

foldl $: (b \rightarrow a \rightarrow b) \rightarrow b \rightarrow \text{Seq } a \rightarrow b$

foldl evalúa de izquierda a derecha.

8.5. La operación reduce

La operación **reduce** se utiliza para reducir una secuencia de valores a un único valor. Recibe como argumento una función que define como se va a combinar cada elemento de la secuencia para obtener un resultado final.

reduce $: (a \rightarrow a \rightarrow a) \rightarrow a \rightarrow \text{Seq } a \rightarrow a$

donde:

- $(a \rightarrow a \rightarrow a)$: toma dos elementos del mismo tipo y devuelve un solo elemento del mismo tipo.
- El segundo argumento a es el elemento neutro. En **reduce** $\oplus e$, el valor e corresponde a la secuencia vacía.
- **Comportamiento para funciones asociativas y no asociativas.**

Si $\oplus$ es asociativa, **reduce** $\oplus e$ s , evalúa a la suma con respecto a $\oplus$ de la secuencia s . Si además e es neutro respecto de $\oplus$, **reduce** coincide con **foldr** $\oplus e$ y **foldl** $\oplus e$.

Si $\oplus$ no es asociativa, obtenemos diferentes resultados dependiendo del orden de reducción.

Limitarse a funciones asociativas no es una buena idea, la suma y multiplicación de punto flotante no es asociativa, por ejemplo.

- **Especificación del orden de reducción.**

Para especificar el comportamiento de **reduce** debemos especificar el orden de reducción. El orden de reducción es parte del TAD y no de la implementación en particular.

8.6. Orden de reducción de reduce

Definiremos la operación **reduce** usando un árbol de combinación para poder especificar el **orden de reducción** de una secuencia de manera clara y eficiente.

Un **árbol de combinación** es una estructura utilizada en algoritmos 'Divide & Conquer' para procesar eficientemente grandes conjuntos de datos. En el contexto del cálculo paralelo y distribuido, los árboles de combinación son especialmente útiles para paralelizar y optimizar operaciones como la reducción, búsqueda y clasificación.

En esencia, un árbol de combinación es un árbol binario que se utiliza para dividir una tarea en partes más pequeñas y luego combinar los resultados de manera eficiente.

```
data Tree a = Leaf a | Node (Tree a) (Tree a)
toTree : Seq a → Tree a
toTree s = case |s| of
  1 → (Leaf s0)
  2 → Node (toTree (take s pp)) (toTree (drop s pp))
where
  pp = 2ilg(n-1)
```

donde *ilg* es el logaritmo entero. El proposito del valor *pp* es encontrar el tamaño más grande de una subsecuencia que equilibra el árbol de combinación. El primer subárbol buscará ser un árbol completo, esto sirve para maximizar la cantidad de operaciones que podemos paralelizar.

Si $|s| = 2^k$, el resultado es un árbol binario perfecto. La operación $2^{\text{ilg}(n-1)}$ devuelve la potencia de 2 más grande que entra en n .

Ahora podemos definir **reduce** sobre árboles

```
reduceT (+) (Leaf x) = x
reduceT (+) (Node l r) = (reduceT (+) l) + (reduceT (+) r)
```

Con esto podemos especificar **reduce** para secuencias, utilizando la vista del árbol:

- Si $|s| = 0$, **reduce** $\oplus b s$ evalúa a b .
- Si $|s| > 0$ y **reduceT** $\oplus (\text{toTree } s)$ evalúa a v , entonces **reduce** $\oplus b s$ evalúa a $(b \oplus v)$.

Observación.

Con esto no estamos dando una implementación de reduce, solo especificando el orden de reducción.

Ejemplo.

Si tenemos la secuencia $s = \langle s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9 \rangle$, entonces (seguir orden con el valor *pp* del logaritmo entero):

$$\text{reduce } \oplus b s = b \oplus (((s_0 \oplus s_1) \oplus (s_2 \oplus s_3) \oplus (s_4 \oplus s_5) \oplus (s_6 \oplus s_7) \oplus (s_8 \oplus s_9)))$$

8.7. Divide and Conquer con reduce

Podemos definir un algoritmo 'Divide & Conquer' genérico como sigue, solo faltaría instanciar los valores de **val**, **base** y **combine**:

```
dyc s = case showt s of
  EMPTY → val
  ELT v → base v
  NODE l r → let (l', r') = dyc l || dyc r
               in combine l' r'
```

Pero esta función la podemos definir en una sola línea utilizando **reduce**:

```
reduce combine val (mal base s)
```

8.8. MCSS usando reduce

La **máxima subsecuencia contigua** es una subsecuencia de números en una lista tal que la suma de sus elementos es máxima entre todas las subsecuencias posibles.

Por ejemplo, si se considera la secuencia de números $\langle -2, 1, -3, 4, -1, 2, 1 - 5, 4 \rangle$, la máxima subsecuencia contigua sería: $\langle 4, -1, 2, 1 \rangle$ ya que la suma de estos elementos es 6, que es la mayor suma posible dentro de todas las subsecuencias contiguas e la secuencia original.

El problema de encontrar la máxima subsecuencia contigua se puede resolver con el patrón 'Divide & Conquer'. La solución hace uso del tuppling: devolver no solo el resultado final, sino otros cálculos parciales que ayudarán al cómputo final. La solución de MCSS devuelve una tupla con:

- El resultado deseado, representa la máxima subsecuencia contigua (el valor final que devolveremos).
- El máximo prefijo: la máxima subsecuencia contigua que termina en la posición actual.
- El máximo sufijo: la máxima subsecuencia contigua que comienza en la posición actual.
- La suma total: la suma total de la subsecuencia actual.

El código de esta solución usando 'Divide & Conquer' sigue la siguiente estructura:

```
val = (0, 0, 0, 0)
base v = let v' = max v 0
        in (v', v', v', v')
combine (m, p, s, t)(m', p', s', t') =
  let resDeseado = max (s + p') m m'
      maxPrefijo  = max p (t + p')
      maxSufijo  = max s' (s + t')
      sumTotal   = t + t'
  in (resDeseado, maxPrefijo, maxSufijo, sumTotal)
```

En lugar de utilizar la función `dyc` definida en la sección anterior, podemos definir la solución en una sola línea suando `reduce`: `reduce combina val (map base s)`

8.9. La operación scan

La operación `scan` es una variante de `reduce` que en lugar de devolver un solo valor, devuelve una secuencia de valores intermedios obtenidos durante el proceso de reducción. Esto permite ver cómo se acumulan los resultados parciales.

Por ejemplo, $\text{scan} \oplus 0 \langle 1, 2, 3 \rangle = (\langle 0, 1, 3, 6 \rangle, 6)$

$$\text{scan} : (a \rightarrow a \rightarrow a) \rightarrow a \rightarrow \text{Seq } a \rightarrow (\text{Seq } a, a)$$

Cuando $\oplus$ es asociativa, `scan  $\oplus$  b s` es equivalente a

$$\text{scan } \oplus b s = (\text{tabulate } (\lambda i \rightarrow \text{reduce } \oplus b (\text{take } s i)), \text{reduce } \oplus b s)$$

El `tabulate` significa: para cada i , con i variando de 0 a n , calcula la suma/operación acumulada hasta el elemento $i - 1$. Por ejemplo, para el índice 1, toma el primer elemento de la secuencia y lo reduce con b , para el segundo índice, toma los dos primeros elementos de la secuencia y los reduce con b .

Notar que esta es una especificación, como implementación sería muy ineficiente. Aparte solo se especifica para $\oplus$ asociativa. Notablemente, `scan` puede implementarse con estos costos:

$$\begin{aligned} W(\text{scan } \oplus l s) &= O(|s|) \\ S(\text{scan } \oplus i s) &= O(\lg |s|) \end{aligned}$$

8.9.1. MCSS usando scan

El problema de la máxima subsecuencia contigua puede escribirse matemáticamente como:

$$\begin{aligned} \text{mcss } s &= \max_{0 \leq i \leq j \leq |s|} \left(\sum_{k=i}^{j-1} s_k \right) \\ &= \max_{0 \leq i \leq j \leq |s|} \left(\sum_{k=0}^{j-1} s_k - \sum_{l=0}^{i-1} s_l \right) \\ &= \max_{0 \leq i \leq j \leq |s|} (X_j - X_i) \text{ donde } X = \text{scan } (+) 0 s \\ &= \max_{0 \leq j \leq |s|} (X_j - \min_{0 \leq i \leq j} X_i) \end{aligned}$$

Esto se puede modelar con un algoritmo utilizando `scan`:

$$\begin{aligned} \text{mcss } s &= \\ &\text{let } x = \text{scan } (+) 0 s \\ &\quad m = \text{scan min } \backslash \text{infty } x \\ &\text{in max } (\text{tabulate } (\lambda j \rightarrow x_j - m_j) |s|) \end{aligned}$$

8.10. Arreglos persistentes

Los **arreglos persistentes** son estructuras de datos que no destruyen el arreglo original al realizar un cambio, sino que lo copian y le agregan la modificación a la copia.

Al igual que los arrays normales, se puede acceder en tiempo constante a cualquier índice. Sin embargo, no se pueden hacer updates destructivos.

Tienen operaciones para crear arreglos a partir de una lista, a partir de funciones y a partir de otros arreglos.

length :: Arr $a \rightarrow \text{Int}$

length p devuelve el tamaño del arreglo p .

nth :: Arr $a \rightarrow \text{Int} \rightarrow a$

nth $p\ i$ es la proyección i -ésima del arreglo p .

fromList :: $[a] \rightarrow \text{Arr } a$

Construye un arreglo a partir de una lista.

tabulate :: $(\text{Int} \rightarrow a) \rightarrow \text{Int} \rightarrow \text{Arr } a$

tabulate $f\ n$ construye un arreglo p de tamaño n tal que

nth $p\ i == f\ i$ para todo $0 \leq i < n$.

subarray :: Arr $a \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Arr } a$

subarray $a\ i\ l$ construye el subarreglo de a que comienza en el índice i y es de longitud l .

8.10.1. Costos de operaciones en arreglos persistentes

Operación	W	S
length p	$O(1)$	$O(1)$
nth $p\ i$	$O(1)$	$O(1)$
fromList xs	$O(xs)$	$O(xs)$
tabulate $f\ n$	$O\left(\sum_{i=0}^n W(f\ i)\right)$	$O\left(\max_{i=0}^n S(f\ i)\right)$
subarray $a\ i\ l$	$O(1)$	$O(1)$

Observaciones.

- La longitud del arreglo se almacena explícitamente junto con el arreglo en la estructura de datos. Esto permite que la operación **length** simplemente acceda a este valor almacenado, en lugar de tener que recalcularlo. Debido a esto, la operación **length** puede ser realizada en tiempo constante $O(1)$.

- La complejidad de la función `subarray` es constante porque en la estructura del arreglo tenemos un puntero al inicio y la longitud del arreglo. Por lo tanto, para obtener un subarreglo, simplemente se cambia el valor de la longitud del arreglo que pasamos como argumento.

8.10.2. Especificación de costo de secuencias basadas en arreglos persistentes

Podemos implementar secuencias usando arreglos persistentes con los siguientes costos:

Operación	W	S
<code>empty</code>	$O(1)$	$O(1)$
<code>singleton</code>	$O(1)$	$O(1)$
<code>length</code>	$O(1)$	$O(1)$
<code>nth</code>	$O(1)$	$O(1)$
<code>take s n</code>	$O(1)$	$O(1)$
<code>drop s n</code>	$O(1)$	$O(1)$
<code>showt s</code>	$O(1)$	$O(1)$
<code>tabulate f n</code>	$O\left(\sum_{i=0}^{n-1} W(f\ i)\right)$	$O\left(\max_{i=0}^{n-1} S(f\ i)\right)$
<code>map f s</code>	$O\left(\sum_{x \in s} W(f\ x)\right)$	$O\left(\max_{x \in s} S(f\ x)\right)$
<code>filter f s</code>	$O\left(\sum_{x \in s} W(f\ x)\right)$	$O\left(\lg(s) + \max_{x \in s} S(f\ x)\right)$
<code>reduce $\oplus$ b s</code>	$O(s)$	$O(\lg(s))$
<code>scan $\oplus$ b s</code>	$O(s)$	$O(\lg(s))$

Para las funciones `tabulate`, `map` y `filter` se ha especificado el costo para cualquier función argumento. Sin embargo, para `reduce` y `scan`, $W(n) \in O(|s|)$ y $S \in O(|s|)$ solo si la función argumento es $O(1)$.

8.10.3. Análisis de costos de reduce en secuencias con arreglos persistentes

En `reduce  $\oplus$  b s`, el orden de reducción afecta el resultado obtenido si la operación $\oplus$ no es asociativa. Si además $\oplus \notin O(1)$, el orden de reducción también afecta los costos.

Consideremos el siguiente algoritmo de ordenación:

```
sort s = reduce merge empty (map singleton s)
```

Veamos un ejemplo. Analicemos el costo de **sort** teniendo los siguientes datos:

- $W(\text{merge } s \ t) = O(|s| + |t|)$
- $S(\text{merge } s \ t) = O(\lg(|s| + |t|))$
- Orden de reducción: $(x_0 \oplus (x_1(\dots \oplus x_{n-1}) \dots))$

Sea $n = |s|$, calculemos el trabajo y la profundidad de **sort**:

- Como en cada paso i , se mezcla una lista de tamaño 1 con otra de tamaño i (por el acumulador creciente). Así, el costo por paso es $O(1 + i)$, por el trabajo de **merge** para dos listas.

Multiplicando por todos los elementos de la secuencia obtenemos la recurrencia:

$$\begin{aligned}
 W(\text{sort } s) &\leq \sum_{i=1}^{n-1} c \cdot (1 + i) \\
 &= c \left(\sum_{i=1}^{n-1} 1 + \sum_{i=1}^{n-1} i \right) \\
 &= c \left((n-1) + \frac{(n-1)n}{2} \right) \\
 &= c \left((n-1) + \frac{n^2 - n}{2} \right) \\
 &= c \cdot \frac{n^2 + n - 2}{2} \in O(n^2)
 \end{aligned}$$

- Como cada **merge** tiene profundidad $O(\lg(1+i))$, la suma de logaritmos nos da la recurrencia obtenida:

$$\begin{aligned}
 S(\text{sort } s) &\leq \sum_{i=1}^{n-1} c \cdot \lg(1 + i) \\
 &= c \cdot (\lg(2) + \lg(3) + \dots + \lg(n)) \\
 &= c \cdot \lg(2 \cdot 3 \cdot \dots \cdot n) \\
 &= c \cdot \lg(n!) \in O(n \lg n)
 \end{aligned}$$

Si en cambio usamos el orden de reducción dado anteriormente con el valor $pp = 2^{\lg(n-1)}$ tenemos una mejora significativa sobre el trabajo del **sort**:

$$W(\text{sort } s) \in O(n \lg n)$$

Conclusión.

El orden de reducción afecta no solo el resultado ($\oplus$ no asociativa), si no también el costo. Para definir el costo en general de **reduce** definimos el conjunto: $\mathcal{O}_r(\text{reduce } \oplus b s) = \{\text{aplicaciones de } \oplus \text{ en el árbol de reducción}\}$ Y usando este conjunto, el costo de **reduce** es:

$$W(\text{reduce } \oplus b s) = O \left(|s| + \sum_{(x \oplus y) \in \mathcal{O}_r(\oplus, b, s)} W(x \oplus y) \right)$$

$$S(\text{reduce } \oplus b s) = O \left(\lg |s| \cdot \max_{(x \oplus y) \in \mathcal{O}_r(\oplus, b, s)} S(x \oplus y) \right)$$

8.10.4. Implementación de scan

Recordemos la función **scan**, es una variante de **reduce** que en lugar de devolver un solo valor, devuelve una secuencia de valores intermedios obtenidos durante el proceso de reducción. Esto permite ver como se acumulan los resultados parciales

$$\text{scan} : (a \rightarrow a \rightarrow a) \rightarrow a \rightarrow \text{Seq } a \rightarrow (\text{Seq } a, a)$$

Cuando $\oplus$ es asociativa, $\text{scan } \oplus b s$ es equivalente a:

$$\text{scan } \oplus b s = (\text{tabulate } (\lambda i \rightarrow \text{reduce } \oplus b (\text{take } s i)), \text{reduce } \oplus b s)$$

Pero esto no es una implementación eficiente, la operación **scan** parece ser inherentemente secuencial ¿Cómo implementarla en paralelo?

Divide & Conquer vs. Contracción

- **Divide & Conquer:** partimos la secuencia en dos y llamamos recursivamente. ¡Pero la mitad derecha depende de la izquierda!
- **Contracción de entrada:** para resolver un problema. A diferencia de Divide & Conquer, hacemos solo una llamada recursiva. Tenemos los siguientes pasos:
 1. Contraemos la instancia del problema a una instancia (mucho) más chica (contracción).
 2. Resolvemos recursivamente la instancia chica.
 3. Usamos la solución de la instancia chica para resolver la grande (expansión).

Este enfoque es útil en algoritmos paralelos: contracción y expansión son usualmente paralelizables y si contraemos una fracción, la recursión tiene profundidad logarítmica.

Ejemplo de contracción en reduce.

1. Consideramos **reduce** + 0 $\langle 2, 3, 6, 2, 1, 4 \rangle$ -
2. Contraemos la secuencia de la siguiente manera: $\langle 2 + 3, 6 + 2, 1 + 4 \rangle = \langle 5, 8, 5 \rangle$.
3. Calculamos recursivamente **reduce** + 0 $\langle 5, 8, 5 \rangle = 18$.
4. En este caso la expansión es la función identidad: el resultado es 18.

Ejemplo de contracción/expansión en scan.

Supongamos que $\oplus$ es asociativa y comparemos la implementación de **scan** (1) con la implementación con contracción y expansión (2):

1. **scan** $\oplus b \langle x_0, x_1, x_2, x_3 \rangle = (\langle b, b \oplus x_0, b \oplus x_0 \oplus x_1 \oplus x_2 \rangle, b \oplus x_0 \oplus x_1 \oplus x_2 \oplus x_3)$
2. **scan** $\oplus b \langle x_0, x_1, x_2, x_3 \rangle = \mathbf{scan} \oplus b \langle x_0 \oplus x_1, x_2 \oplus x_3 \rangle = (\langle b, b \oplus x_0 \oplus x_1 \rangle, b \oplus x_0 \oplus x_1 \oplus x_2 \oplus x_3)$

Implementación de contracción/expansión en scan.

Para la implementación, buscamos que dada como entrada la secuencia s y el resultado s' del llamado recursivo sobre la contracción, obtener r como sigue

- $s = \langle x_0, x_1, x_2, x_3 \rangle$
- $s' = (\langle b, b \oplus x_0 \oplus x_1 \rangle, b \oplus x_0 \oplus x_1 \oplus x_2 \oplus x_3)$
- $r = (\langle b, b \oplus x_0, b \oplus x_0 \oplus x_1, b \oplus x_0 \oplus x_1 \oplus x_2 \rangle, b \oplus x_0 \oplus x_1 \oplus x_2 \oplus x_3)$

El siguiente proceso nos da un algoritmo paralelo para implementar **scan** y también fija el orden de reducción:

1. El total (segunda componente del par que devuelve **scan**) lo obtenemos del resultado del llamado recursivo s' .
2. Para construir la secuencia (primera componente del par que devuelve **scan**) se sigue el siguiente proceso, donde r_i son elementos de la secuencia final y s'_i de la secuencia del llamado recursivo contraído:

$$a) \ r_i = s'_{\frac{i}{2}} \text{ si } i \text{ es par.}$$

$$b) \ r_i = s'_{\lfloor \frac{i}{2} \rfloor} + s_{i-1}$$

Observar en r lo siguiente: $r_0 = s'_0 = b$ (i impar), $r_1 = s'_0 + s_0 = b + x_0$ (i impar), $r_2 = s'_1 = b \oplus x_0 \oplus x_1$ (i par).

Orden de reducción de scan.

Si la operación $\oplus$ no es asociativa el orden de reducción es importante. Con el algoritmo anteriormente descrito tenemos el siguiente orden de reducción que no coincide con el orden de reducción de **reduce** sobre los prefijos:

$$\begin{aligned} \text{scan} \oplus b \langle x_0, x_1, x_2, x_3, x_4, x_5 \rangle = & (\langle b, \\ & b \oplus x_0, \\ & b \oplus (x_0 \oplus x_1), \\ & (b \oplus (x_0 \oplus x_1)) \oplus x_2, \\ & b \oplus ((x_0 \oplus x_1) \oplus (x_2 \oplus x_3)), \\ & (b \oplus ((x_0 \oplus x_1) \oplus (x_2 \oplus x_3))) \oplus x_4, \\ & b \oplus (((x_0 \oplus x_1) \oplus (x_2 \oplus x_3)) \oplus (x_4 \oplus x_5)) \rangle) \end{aligned}$$

Costo de scan.

Calculamos costos para el scan en la implementación con arreglos. Suponemos que $\oplus$ tiene costo constante. Los costos de **scan** son:

- $W(n) = W(\frac{n}{2}) + kn \Rightarrow W(n) \in O(n)$
- $S(n) = S(\frac{n}{2}) + k \Rightarrow S(n) \in O(\lg n)$

Y definiendo el conjunto de aplicaciones de $\oplus$ en **scan** como:

$$\mathcal{O}_s(\text{scan} \oplus b \ s) = \{\text{aplicaciones de } \oplus \text{ en el árbol de reducción}\}$$

llegamos a los siguientes costos para **scan**:

$$\begin{aligned} W(\text{scan} \oplus b \ s) &= O \left(|s| + \sum_{(x \oplus y) \in \mathcal{O}_r(\oplus, b, s)} W(x \oplus y) \right) \\ S(\text{scan} \oplus b \ s) &= O \left(\lg |s| \cdot \max_{(x \oplus y) \in \mathcal{O}_r(\oplus, b, s)} S(x \oplus y) \right) \end{aligned}$$

8.11. Más funciones de alto orden para secuencias

8.11.1. La operación collect

Supongamos que trabajamos con secuencias **Seq** a , donde existe un orden total para los elementos en a . Esto induce un orden lexicográfico sobre secuencias.

Sea a un tipo con un orden total (el tipo de las claves) y sea b un tipo cualquiera (el tipo de los datos), entonces la función

$$\text{collect} : \text{Seq} \ (a, b) \rightarrow \text{Seq} \ (a, \text{Seq} \ b)$$

recolecta todos los datos asociados a cada clave. La secuencia estará ordenada según el orden de a .

Por ejemplo:

$$= \text{collect } \langle (3, \text{'EDyAII'}), (1, \text{'Prog1'}), (2, \text{'Prog2'}), (2, \text{'EDyAI'}), (3, \text{'SOI'}) \rangle$$

$$= \langle (1, \langle \text{'Prog1'} \rangle), (2, \langle \text{'Prog2'}, \text{'EDyAI'} \rangle), (3, \langle \text{'EDyAII'}, \text{'SOI'} \rangle) \rangle$$

8.11.2. Implementación de collect

`collect` se puede implementar en dos pasos:

- Ordenar la entrada según las claves. Esto tiene como efecto juntar claves iguales.
- Juntar todos los valores de claves iguales.

Y el costo de `collect` se puede dividir en

- Ordenar tiene $W(n) \in O(W_c \cdot n \lg n)$ y $S(n) \in (S_n \cdot \lg(n)^2)$, donde W_c y S_c son el trabajo y profundidad de la comparación de claves.
- Juntar todas las claves es $W(n) \in O(n)$ y $S(n) \in O(\lg n)$

Por lo tanto, el costo del `collect` está dominado por el costo de la ordenación.

8.11.3. operación MAP/COLLECT/REDUCE: map-reduce de Google

Esta operación implica hacer un `map`, seguido de `collect`, seguido de varios `reduce`.

Ejemplo: asociar a cada palabra clave una secuencia de documentos donde esta aparece.

- `map apv` se aplica a una secuencia de documentos donde `apv` transforma cada documento en secuencias de pares clave/valor.
- Esto último devuelve una secuencia de secuencias de pares clave/valor que se aplana con `join`.
- Luego, se aplica un `collect` para combinar las claves con todos los documentos donde esta aparece.
- Luego, cada par de clave y secuencia de valores es 'reducido' a un resultado por `red`.

Generalmente `apv` y `red` son funciones secuenciales. El paralelismo viene de aplicarlas en un `map`.

```
mapCollectReduce apv red s =  
  let pairs = join (map apv s)  
      groups = collect pairs  
  in map red groups
```

Ejemplo.

¿Cuántas veces aparece una palabra en una colección de documentos?

```
countWords s = mapCollectReduce apv red s
```

Esta función aplica `words` sobre el documento d , que devuelve una secuencia de palabras separadas por espacios. Luego, a través del `map`, asigna el par (palabra, 1) a cada palabra devuelta por `words`:

```
apv d = map ( $\lambda w \rightarrow (w, 1)$ ) (words d)
```

Como luego se aplica un `join (map apvs s)` en `mapCollectReduce`, se aplana esta secuencia de pares. Luego, con el `collect` se recolectan pares (palabra, secuenciaDeUnos), donde la `secuenciaDeUnos` tendrá un elemento por cada aparición de la palabra. Es decir, `length s = cantVeces Palabra`.

Con el `map red groups`, sumamos la cantidad de unos que aparece en cada secuencia del par (palabra, secuenciaDeUnos) para obtener finalmente una secuencia de pares (palabra, cantVecesAparece).

```
red (w, s) = (w, reduce (+) 0 s)
```